

Modeling the dynamic heat inactivation of *Vibrio parahaemolyticus* in oyster digestive diverticula

Weifan Wu, et al.

2026-07-08

Table of contents

1. Global Environment & Session Info	2
2. Data Import.....	5
3. Statistical comparison between two different media	8
4. Data Visualization.....	9
4.1 Inactivation data facet by both target temp and strain	9
4.1.1 Statistical comparison of 2 strains.....	10
4.2 Inactivation data with strain combined	14
4.3 time-temperature profile	15
5. Data preprocessing for modeling	17
5.1 Checking distribution of count	18
6. Functions for modeling	22
6.1 Model Fitting	22
7. Modeling.....	26
7.0.1 Decimal reduction time calculation	28
7.0.2 Prediction of reduction	30
7.0.3 Plot using the best parameter set with observed value overlaid.....	32
7.0.4 Goodness-of-fit check.....	34
7.0.5 Bacterial reduction estimation given different initial conc.....	35
7.1 Plot for publication	35
7.2 Geeraerd	39
7.3 Geeraerd with no shoulder	42
7.3.1 Plot using the best parameter set with observed value overlaid (with shoulder) 44	
7.3.2 Plot using the best parameter set with observed value overlaid (with no shoulder)	
.....	45

7.3.3 Goodness-of-fit check (with shoulder)	47
7.3.4 Goodness-of-fit check (with no shoulder).....	48
7.4 Log-linear (Bigelow)	48
7.4.1 Plot using the best parameter set with observed value overlaid.....	50
7.4.2 Goodness-of-fit check.....	51

1. Global Environment & Session Info

```
# Set Seed
set.seed(5025)
# Limit the number of significant digits to 3
options(digits = 3)

# Packages
## Data Entry and Manipulation
library(tidyverse)

— Attaching core tidyverse packages ————— tidyverse 2.0.0
—
✓ dplyr      1.2.1      ✓ readr      2.2.0
✓ forcats    1.0.1      ✓ stringr    1.6.0
✓ ggplot2    4.0.3      ✓ tibble     3.3.1
✓ lubridate  1.9.5      ✓ tidyr      1.3.2
✓ purrr      1.2.2
— Conflicts ————— tidyverse_conflicts()
—
✗ dplyr::filter() masks stats::filter()
✗ dplyr::lag()     masks stats::lag()
! Use the conflicted package (<http://conflicted.r-lib.org/>) to force all co
nflicts to become errors

library(here)

here() starts at /Users/weifan.wu/Library/CloudStorage/Dropbox-UF/Weifan Wu/N
oVOysR/WW/Vibrio_Project

library(readxl)
library(skimr)
library(nlme)
```

Attaching package: 'nlme'

The following object is masked from 'package:dplyr':

collapse

```
library(glmTMB)
library(fitdistrplus)
```

Loading required package: MASS

Attaching package: 'MASS'

The following object is masked from 'package:dplyr':

select

Loading required package: survival

```
## Bioinactivation
library(bioinactivation)
library(mvtnorm) # For calc CI and PI for graphs
library(FME) # for performing global fit on dynamic data
```

Loading required package: deSolve
Loading required package: rootSolve
Loading required package: coda

```
library(cowplot)
```

Attaching package: 'cowplot'

The following object is masked from 'package:lubridate':

stamp

```
library(emmeans)
```

Welcome to emmeans.
Caution: You lose important information if you filter this package's results.
See '? untidy'

```
## Data Visualization
library(patchwork)
```

Attaching package: 'patchwork'

The following object is masked from 'package:cowplot':

align_plots

The following object is masked from 'package:MASS':

area

```
## Session Info
```

```
library(sessioninfo)
```

```
sessioninfo::session_info (pkgs = c("attached"))
```

```
— Session info —
```

```
setting  value
version  R version 4.6.1 (2026-06-24)
os       macOS Tahoe 26.4
system   aarch64, darwin23
ui       X11
language (EN)
collate  en_US.UTF-8
ctype    en_US.UTF-8
tz       America/New_York
date     2026-07-08
pandoc    3.8.3 @ /Applications/RStudio.app/Contents/Resources/app/quarto/bin
/tools/aarch64/ (via rmarkdown)
quarto    1.9.37 @ /Applications/RStudio.app/Contents/Resources/app/quarto/bin/quarto
```

```
— Packages —
```

```
package      * version    date (UTC) lib source
bioinactivation * 1.3.1      2025-11-03 [1] CRAN (R 4.6.0)
coda          * 0.19-4.1   2024-01-31 [1] CRAN (R 4.6.0)
cowplot       * 1.2.0      2025-07-07 [1] CRAN (R 4.6.0)
deSolve       * 1.42       2026-03-20 [1] CRAN (R 4.6.0)
dplyr         * 1.2.1      2026-04-03 [1] CRAN (R 4.6.0)
emmeans       * 2.0.3      2026-04-09 [1] CRAN (R 4.6.0)
fitdistrplus  * 1.2-6      2026-01-24 [1] CRAN (R 4.6.0)
FME           * 1.3.6.4    2025-07-25 [1] CRAN (R 4.6.0)
forcats       * 1.0.1      2025-09-25 [1] CRAN (R 4.6.0)
ggplot2       * 4.0.3      2026-04-22 [1] CRAN (R 4.6.0)
glmmTMB       * 1.1.14     2026-01-15 [1] CRAN (R 4.6.0)
here          * 1.0.2      2025-09-15 [1] CRAN (R 4.6.0)
lubridate     * 1.9.5      2026-02-04 [1] CRAN (R 4.6.0)
MASS          * 7.3-65     2025-02-28 [1] CRAN (R 4.6.1)
mvtnorm       * 1.4-1      2026-06-06 [1] CRAN (R 4.6.0)
nlme          * 3.1-169    2026-03-27 [1] CRAN (R 4.6.1)
patchwork     * 1.3.2      2025-08-25 [1] CRAN (R 4.6.0)
purrr         * 1.2.2      2026-04-10 [1] CRAN (R 4.6.0)
readr         * 2.2.0      2026-02-19 [1] CRAN (R 4.6.0)
readxl        * 1.5.0      2026-05-16 [1] CRAN (R 4.6.0)
rootSolve     * 1.8.2.4    2023-09-21 [1] CRAN (R 4.6.0)
sessioninfo   * 1.2.4      2026-06-04 [1] CRAN (R 4.6.0)
```

```

skimr          * 2.2.2      2026-01-10 [1] CRAN (R 4.6.0)
stringr        * 1.6.0      2025-11-04 [1] CRAN (R 4.6.0)
survival       * 3.8-6       2026-01-16 [1] CRAN (R 4.6.1)
tibble         * 3.3.1      2026-01-11 [1] CRAN (R 4.6.0)
tidyr          * 1.3.2      2025-12-19 [1] CRAN (R 4.6.0)
tidyverse      * 2.0.0      2023-02-22 [1] CRAN (R 4.6.0)

```

```

[1] /Library/Frameworks/R.framework/Versions/4.6/Resources/library
* — Packages attached to the search path.

```

—

```

knitr::opts_chunk$set(error = TRUE, warning = FALSE, message = FALSE)

quiet_predict_inactivation <- function(...) {
  result <- NULL
  invisible(capture.output(
    result <- suppressWarnings(bioinactivation::predict_inactivation(...)),
    type = "output"
  ))
  result
}

```

2. Data Import

```

df_dyn <- readRDS(here("Data", "df_dyn.RDS")) %>%
  mutate(
    # CFU and plated_ml are stored as the total count and volume across two p
    lates.
    count = as.integer(round(cfu)),
    plate_fraction = plated_ml * dilution,
    lod_log_cfu = log10(1 / plate_fraction),
    no_colonies = count == 0L,
    log_cfu_plot = if_else(no_colonies, 0.5, as.numeric(log_cfu))
  )

```

```

LOD <- df_dyn %>%
  filter(dilution == max(dilution, na.rm = TRUE)) %>%
  summarise(lod = min(lod_log_cfu, na.rm = TRUE), .groups = "drop") %>%
  pull(lod)

```

```
df_comeup <- readRDS (here("Data", "df_comeup.RDS"))
```

```
skim(df_dyn)
```

Data summary

```
Name          df_dyn
```

Number of rows 132
 Number of columns 13

Column type frequency:

factor 2
 logical 1
 numeric 10

Group variables None

Variable type: factor

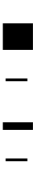
skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
temp_target_c	0	1	FALSE	3	98: 48, 63: 42, 74: 42
strain	0	1	FALSE	2	FDA: 66, TX_: 66

Variable type: logical

skim_variable	n_missing	complete_rate	mean	count
no_colonies	0	1	0.33	FAL: 89, TRU: 43

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
time_min	0	1	5.14	3.72	0.0	2.0	4.5	8.00	12.0	
					0	0	0		0	
										
										
										
log_cfu	0	1	3.20	1.48	1.7	1.7	2.8	4.28	6.09	
										
										
										
										
rep	0	1	2.00	0.82	1.0	1.0	2.0	3.00	3.00	
					0	0	0			
										

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
cfu	0	1	92.63	129.08	0.00	0.00	15.50	159.25	496.00	
dilution	0	1	0.14	0.09	0.00	0.02	0.20	0.20	0.20	
plated_ml	0	1	0.20	0.00	0.20	0.20	0.20	0.20	0.20	
count	0	1	92.63	129.08	0.00	0.00	15.50	159.25	496.00	
plate_fraction	0	1	0.03	0.02	0.00	0.00	0.04	0.04	0.04	
lod_log_cfu	0	1	1.90	0.76	1.40	1.40	1.40	2.40	3.40	
log_cfu_plot	0	1	2.81	1.92	0.50	0.50	2.89	4.28	6.09	

skim(df_comeup)

Data summary

Name	df_comeup
Number of rows	793
Number of columns	4

Column type frequency:

factor	2
numeric	2

Group variables	None
-----------------	------

Variable type: factor

skim_variable	n_missin g	complete_rat e	ordere d	n_uniqu e	top_counts
temp_target_ c	0	1	FALSE	3	63: 297, 74: 296, 98: 200
strain	0	1	FALSE	2	VP3: 400, VP2: 393

Variable type: numeric

skim_variable	n_missi ng	complete_r ate	me an	sd	p0	p25	p5 0	p75	p1 00	hist
time_min	0	1	5.6 3	3.4 3	0. 0	2.7 5	5. 5	8.2 5	12 .3	
temp_recorded_c _ave	0	1	68. 54	17. 71	20 .4	61. 80	65 .2	74. 40	98 .8	

3. Statistical comparison between two different media

```
df_media <- read_excel(here("Data", "Vibrio Parahaemolyticus on PBS at 74C April 2026.xlsx"), sheet = 3) %>%  
  mutate(plate = as.factor(plate)) %>%  
  filter(plate != "TCBS")
```

```
# Model with strain-by-temperature fixed effect
```



```
fit_media_lm <- lm(log_cfu ~ plate * time, data = df_media)

emm <- emmeans(fit_media_lm, ~ plate)

contrast_df <- as.data.frame(
  summary(pairs(emm), infer = TRUE)
)
contrast_df
```

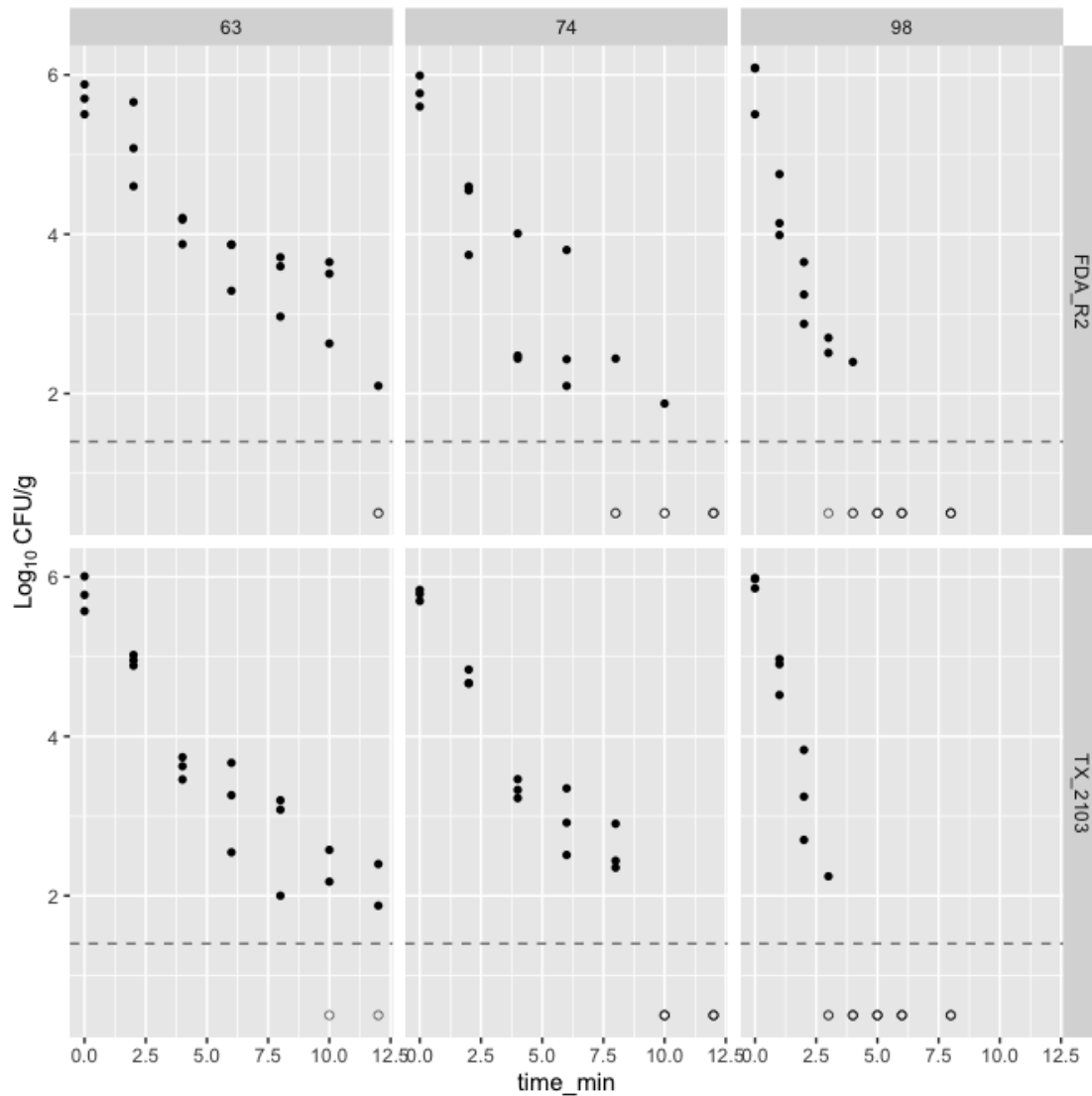
contrast	estimate	SE	df	lower.CL	upper.CL	t.ratio	p.value
ChromAgar - TSA	-0.326	0.304	14	-0.979	0.327	-1.072	0.3019

Confidence level used: 0.95

4. Data Visualization

4.1 Inactivation data facet by both target temp and strain

```
df_dyn %>%
  ggplot(aes(x = time_min, y = log_cfu_plot)) +
  geom_hline(
    yintercept = LOD,
    linetype = "dashed",
    color = "gray50"
  ) +
  geom_point(data = ~ filter(.x, !no_colonies), shape = 16, color = "black")
+
  geom_point(data = ~ filter(.x, no_colonies), shape = 1, color = "black", alpha = 0.5) +
  labs(y = expression(Log[10]~"CFU/g")) +
  facet_grid(strain~temp_target_c)
```



4.1.1 Statistical comparison of 2 strains

```
# Negative-binomial GLMM for colony counts.
# No-colony observations remain count = 0.
df_dyn_test <- df_dyn %>%
  mutate(
    cond = interaction(temp_target_c, time_min, drop = TRUE)
  )

# Model with strain-by-temperature fixed effect.
fit <- glmmTMB(
  count ~ strain * temp_target_c + offset(log(plate_fraction)) + (1 | cond),
  family = nbinom2,
  data = df_dyn_test
)

drop1(fit, test = "Chisq")
```

Single term deletions

Model:

```
count ~ strain * temp_target_c + offset(log(plate_fraction)) +  
  (1 | cond)
```

	Df	AIC	LRT	Pr(>Chi)
<none>	1157			
strain:temp_target_c	2	1156	3.02	0.22

Estimated strain contrasts within each target temperature. The NB model uses

a natural-log link, so divide by log(10) to report the contrast on a log10 scale.

```
emm <- emmeans(fit, ~ strain | temp_target_c)
```

```
contrast_df <- as.data.frame(  
  summary(pairs(emm), infer = TRUE)  
) %>%
```

```
  rename(  
    lower.CL = any_of("asympt.LCL"),  
    upper.CL = any_of("asympt.UCL")  
  ) %>%
```

```
  mutate(  
    estimate_log10 = estimate / log(10),  
    lower_log10 = lower.CL / log(10),  
    upper_log10 = upper.CL / log(10)  
  )
```

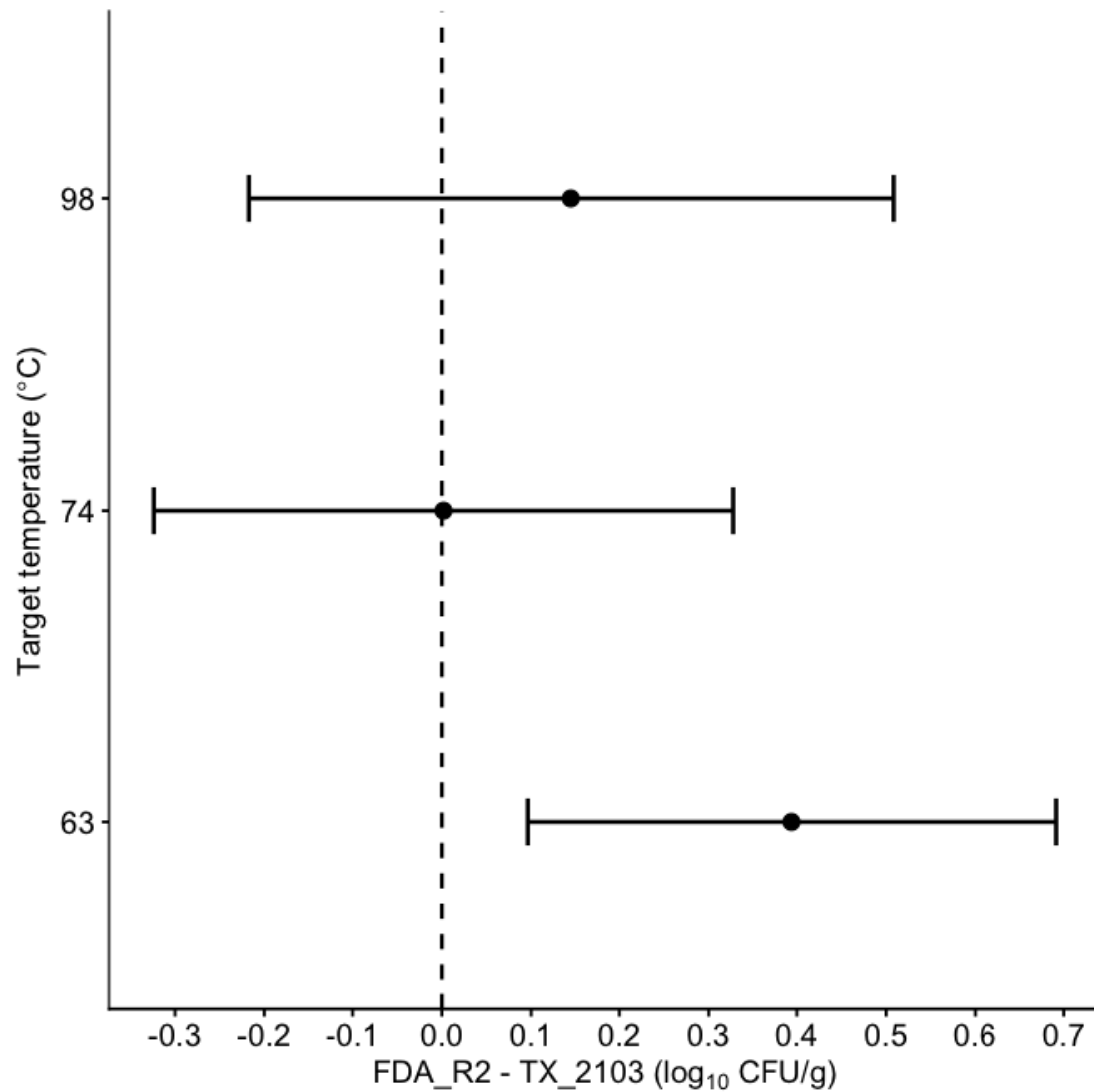
```
contrast_df
```

	contrast	temp_target_c	estimate	SE	df	lower.CL	upper.CL	z.ratio
1	FDA_R2 - TX_2103	63	0.90708	0.350	Inf	0.222	1.592	2.5942
2	FDA_R2 - TX_2103	74	0.00429	0.383	Inf	-0.746	0.754	0.0112
3	FDA_R2 - TX_2103	98	0.33514	0.426	Inf	-0.500	1.171	0.7863
	p.value	estimate_log10	lower_log10	upper_log10				
1	0.00948	0.39394	0.0963	0.692				
2	0.99105	0.00186	-0.3238	0.327				
3	0.43171	0.14555	-0.2173	0.508				

```
plot_strain_temp <- ggplot(contrast_df, aes(x = estimate_log10, y = temp_target_c)) +
```

```
  geom_vline(xintercept = 0, linetype = "dashed", linewidth = 0.8) +  
  geom_errorbar(aes(xmin = lower_log10, xmax = upper_log10), orientation = "y",  
    width = 0.15, linewidth = 0.9) +  
  scale_x_continuous(breaks = seq(-1, 1, by = 0.1)) +  
  geom_point(size = 3) +  
  labs(x = expression("FDA_R2 - TX_2103 (*log[10]*" CFU/g)"),  
    y = expression("Target temperature (*degree*C)")) +  
  theme_classic(base_size = 14) +  
  theme(axis.text=element_text(size=14,color="black"))
```

```
plot_strain_temp
```



```
df_63 <- df_dyn %>%
  filter(temp_target_c == 63) %>%
  mutate(
    time_min = factor(time_min)
  )

fit_63 <- glmmTMB(
  count ~ strain * time_min + offset(log(plate_fraction)) + (1 | rep),
  family = nbinom2,
  data = df_63
)

drop1(fit_63, test = "Chisq")

Single term deletions
```

```

Model:
count ~ strain * time_min + offset(log(plate_fraction)) + (1 |
  rep)
              Df AIC  LRT Pr(>Chi)
<none>              449
strain:time_min  6 455 18.5    0.005 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

emm_63 <- emmeans(fit_63, ~ strain | time_min)

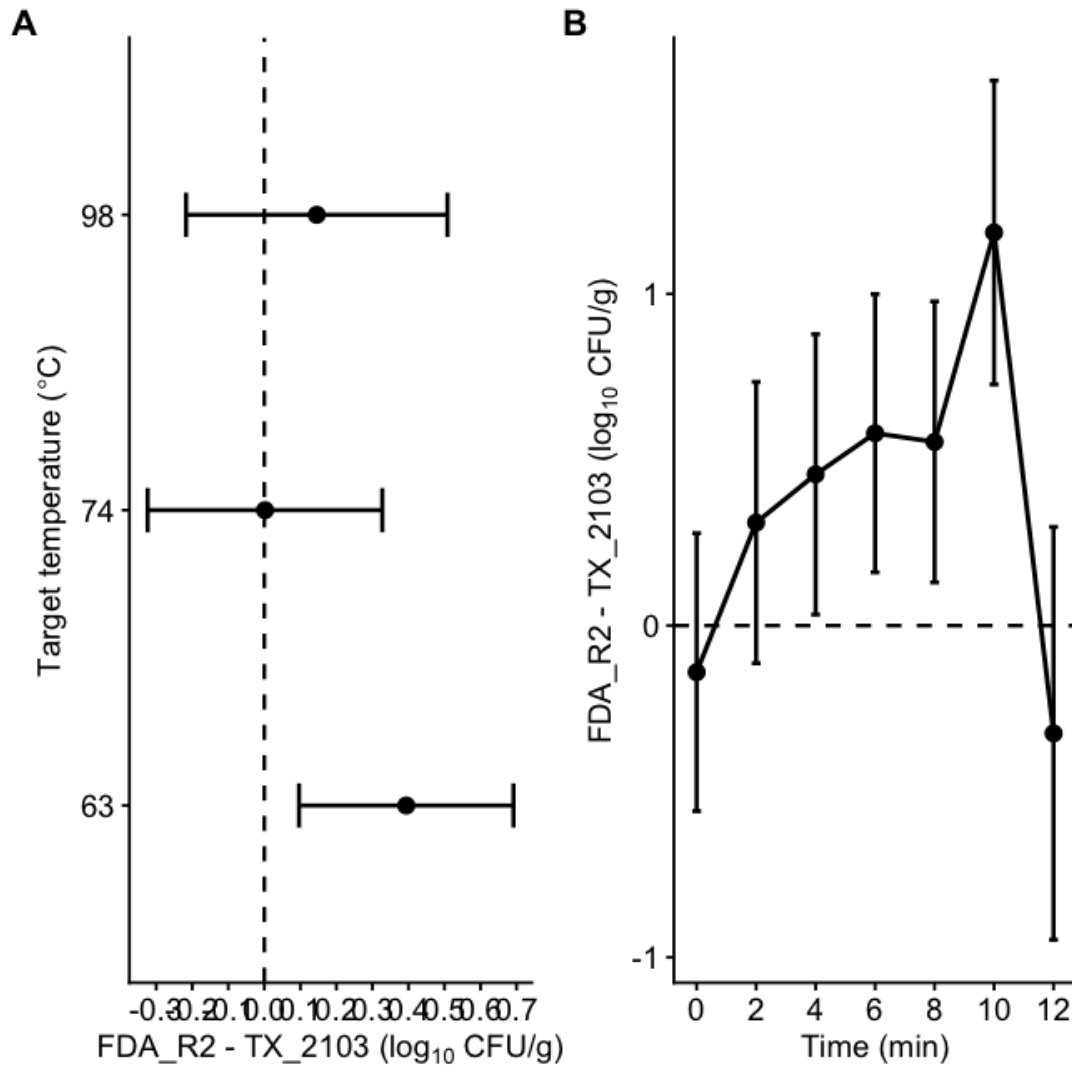
contrast_63 <- as.data.frame(
  summary(pairs(emm_63), infer = TRUE)
) %>%
  rename(
    lower.CL = any_of("asyp.LCL"),
    upper.CL = any_of("asyp.UCL")
  ) %>%
  mutate(
    estimate_log10 = estimate / log(10),
    lower_log10 = lower.CL / log(10),
    upper_log10 = upper.CL / log(10)
  )

plot_strain_time <- ggplot(contrast_63, aes(x = as.numeric(as.character(time_
min)), y = estimate_log10)) +
  geom_hline(yintercept = 0, linetype = "dashed", linewidth = 0.8) +
  geom_errorbar(aes(ymin = lower_log10, ymax = upper_log10), width = 0.3, lin
ewidth = 0.9) +
  geom_point(size = 3) +
  geom_line(linewidth = 1) +
  scale_x_continuous(breaks = seq(0, 12, 2)) +
  labs(
    x = "Time (min)",
    y = expression("FDA_R2 - TX_2103 ("*log[10]*" CFU/g)")
  ) +
  theme_classic(base_size = 14)+
  theme(axis.text=element_text(size=14,color="black"))

combined_plot <- plot_strain_temp + plot_strain_time +
  plot_annotation(tag_levels = "A") &
  theme(
    plot.tag = element_text(size = 16, face = "bold")
  )

combined_plot

```



```
ggsave(
  here("Plot", "Figure_strain_comparison_combined.tiff"),
  plot = combined_plot,
  width = 11,
  height = 5,
  units = "in",
  dpi = 600,
  compression = "lzw"
)
```

The count-based negative-binomial GLMM did not indicate a significant difference between VP2 and VP3. Strain variable will be removed.

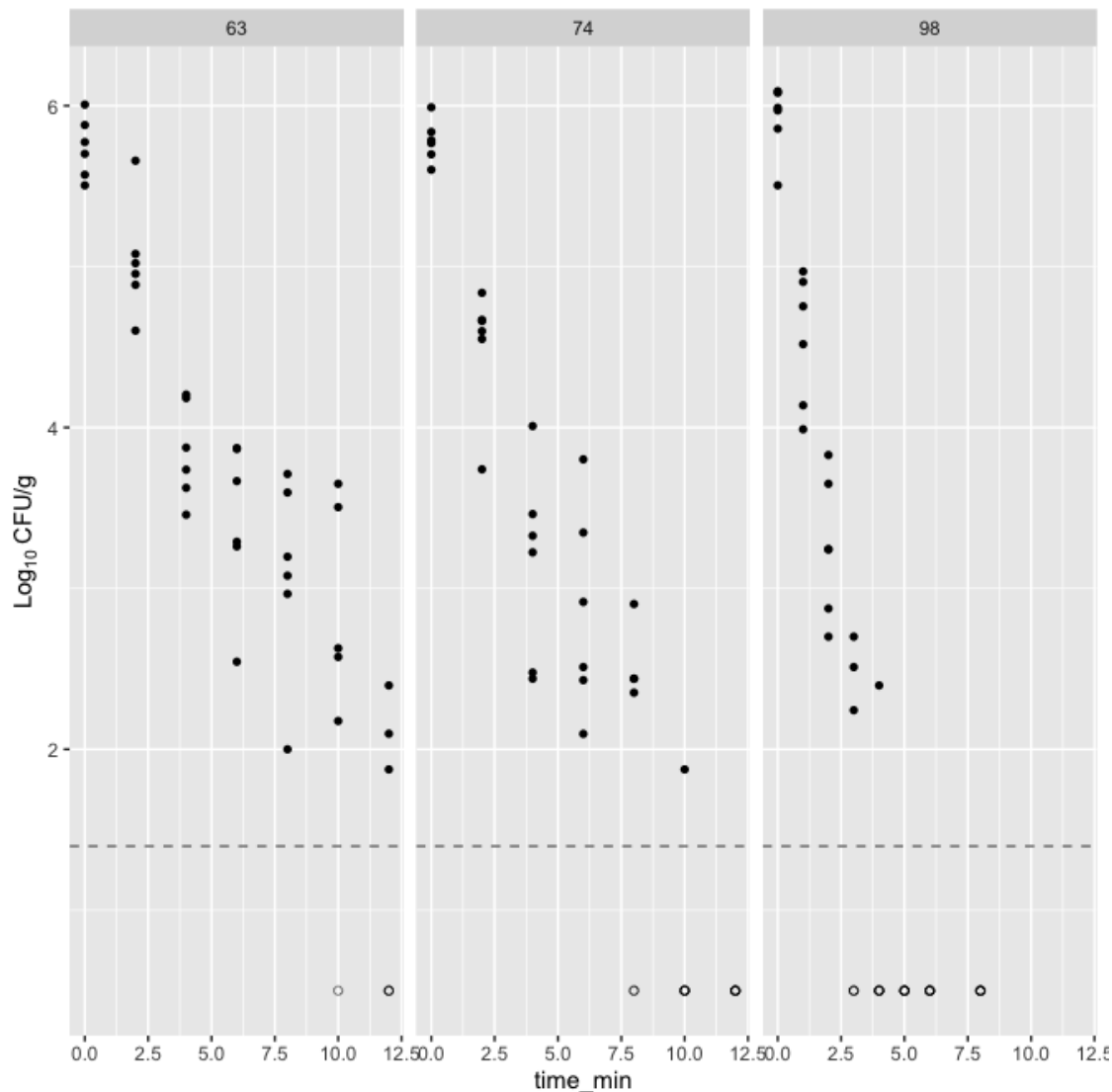
4.2 Inactivation data with strain combined

```
df_dyn %>%
  ggplot(aes(x = time_min, y = log_cfu_plot)) +
```

```

geom_hline(
  yintercept = LOD,
  linetype = "dashed",
  color = "gray50"
) +
geom_point(data = ~ filter(.x, !no_colonies), shape = 16, color = "black")
+
geom_point(data = ~ filter(.x, no_colonies), shape = 1, color = "black", al
pha = 0.5) +
labs(y = expression(Log[10]~"CFU/g")) +
facet_grid(~temp_target_c)

```



4.3 time-temperature profile

```

df_temp <- df_comeup %>%
  group_by(temp_target_c, time_min) %>%
  summarise(temp_mean = mean(temp_recorded_c_ave),

```

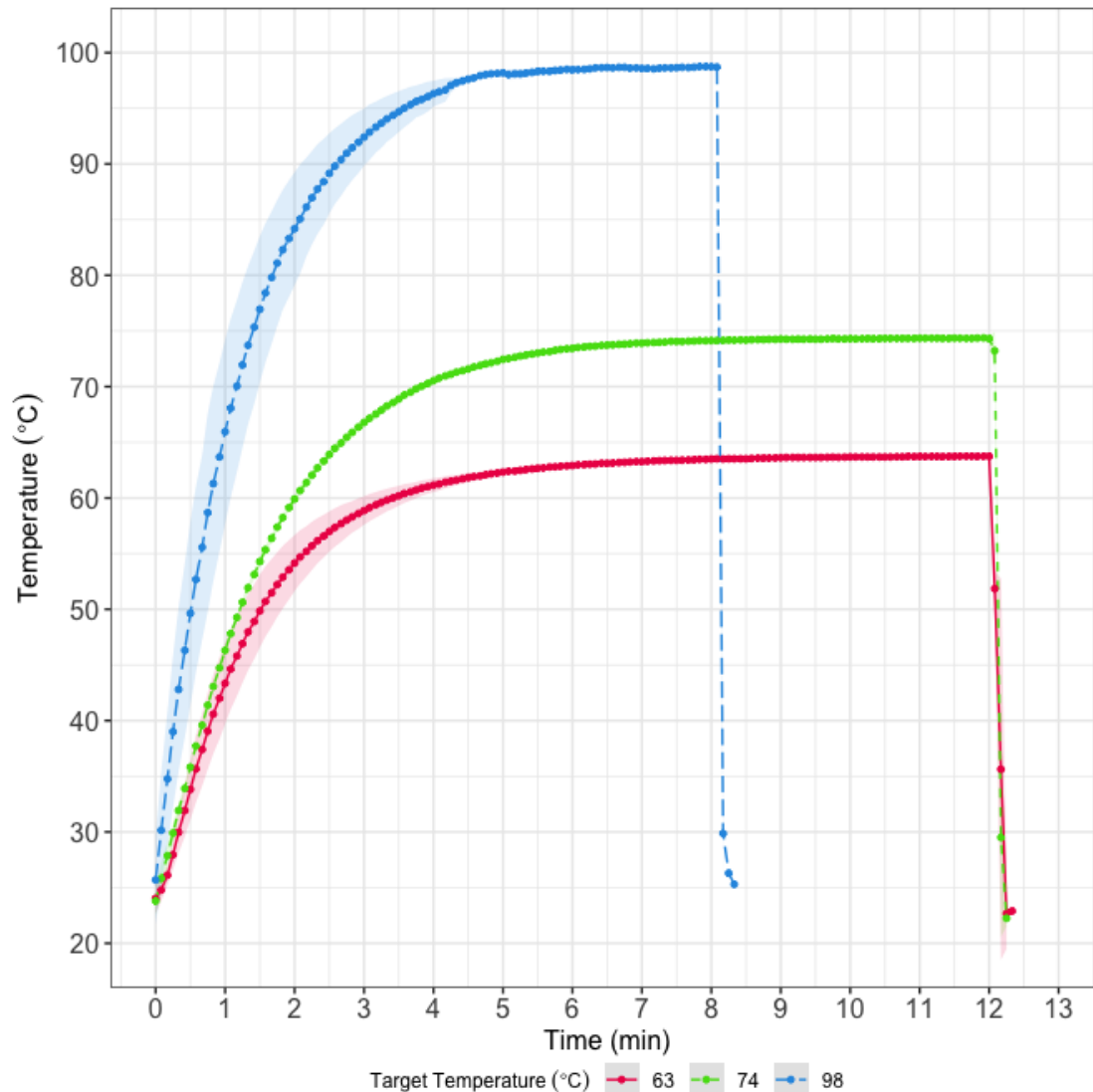
```

    temp_sd = sd(temp_recorded_c_ave), .groups = "drop")

p_temp <- df_temp %>%
  ggplot(aes(
    x = time_min, y = temp_mean,
    color = temp_target_c,
    linetype = temp_target_c,
    group = temp_target_c
  )) +
  # Mean and SD ribbon
  geom_ribbon(aes(
    ymin = temp_mean - temp_sd,
    ymax = temp_mean + temp_sd,
    fill = temp_target_c
  ),
  alpha = 0.15, color = NA) +
  geom_line(size = 0.5) +
  geom_point(size = 1) +
  labs(
    y = bquote(Temperature~(degree*C)),
    x = "Time (min)",
    color = bquote(Target~Temperature~(degree*C)),
    linetype = bquote(Target~Temperature~(degree*C))
  ) +
  scale_color_manual(values = c("63" = "#ED0F54", "74" = "#52DD12", "98" = "#
2998E2")) +
  scale_fill_manual(values = c("63" = "#ED0F54", "74" = "#52DD12", "98" = "#
2998E2")) +
  scale_linetype_manual(values = c("63" = "solid", "74" = "dashed", "98" = "l
ongdash")) +
  scale_x_continuous(breaks = seq(0, 13, by = 1)) +
  scale_y_continuous(breaks = seq(20, 100, by = 10)) +
  coord_cartesian(xlim = c(0, 13), ylim = c(20, 100)) +
  theme_bw() +
  theme(
    legend.position = "bottom",
    legend.justification = "center",
    legend.margin = margin(t = 0, r = 0, b = 0, l = 0),
    legend.box.margin = margin(t = -8, r = 0, b = 0, l = 0),
    legend.title = element_text(size = 9),
    legend.text = element_text(size = 9),
    axis.title = element_text(size = 12),
    axis.text = element_text(size = 12)) +
  guides(fill = "none")

p_temp

```

```
ggsave(file = here("Plot", "ComeUp_Temp_Plot.tiff"), plot = p_temp, width = 6,
height = 4, units = "in", dpi = 400)
```

5. Data preprocessing for modeling

```
# Left join the come up time df to get the measured temperature
df_dyn2 <- df_dyn %>%
  left_join(
    df_temp %>% dplyr::select(temp_target_c, time_min, temp_mean),
    by = c("temp_target_c", "time_min")
  )

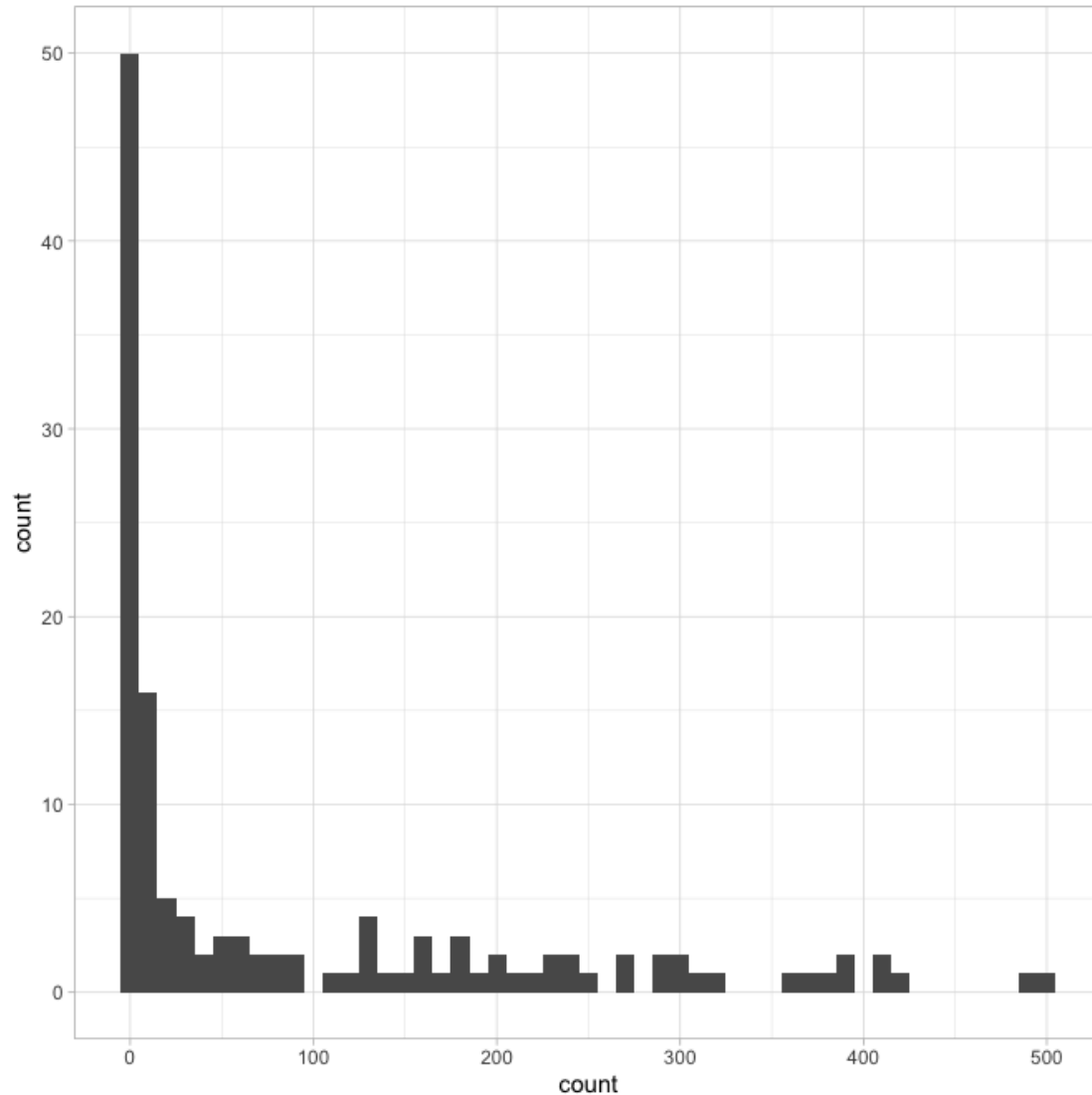
saveRDS (df_dyn2, here("Data", "df_dyn2.RDS"))
```

The CFU column is stored as the total colony count from two 0.1 mL plates, with plated_ml = 0.2. Observations with no colonies on the lowest-dilution plates are

modeled as `count = 0`; `log_cfu_plot` is used only for visualization, placing those observations below the effective two-plate LOD on log-scale plots.

5.1 Checking distribution of count

```
df_dyn2 <- readRDS(here("Data", "df_dyn2.RDS"))
# start with ggplot
ggplot(df_dyn2, aes(count)) +
  geom_histogram(binwidth = 10) +
  theme_light()
```



```
# Check if variance = mean
g <- df_dyn2 %>%
  group_by(temp_target_c, time_min, dilution, plated_ml) %>%
  summarise(
    n = n(),
    mean = mean(count, na.rm = TRUE),
```

```

    var = var(count, na.rm = TRUE),
    vmr = var / mean, # variance-to-mean ratio
    .groups = "drop"
)

summary(g$vmr)

  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   NAs
  3.00   9.76   33.93   52.58   96.27  190.37    4

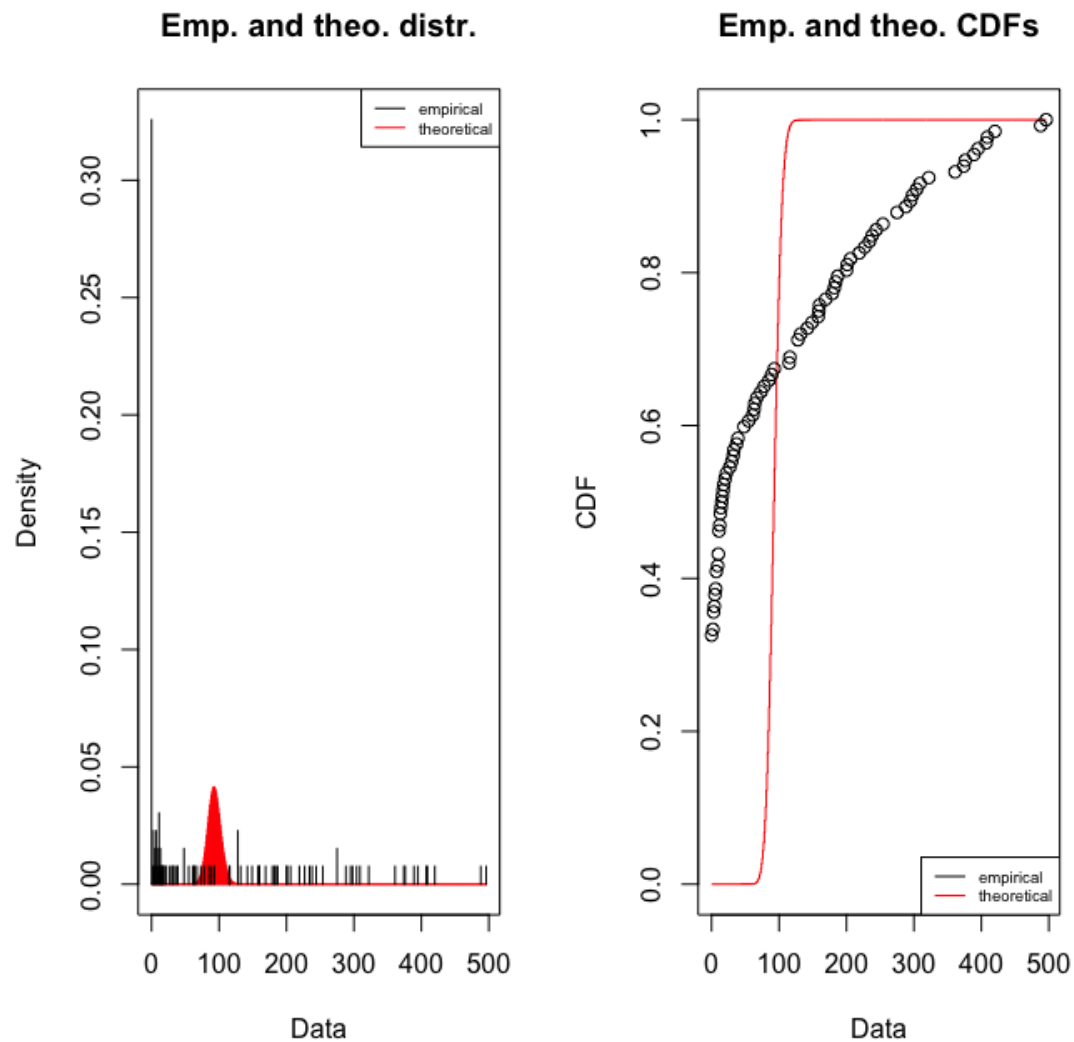
quantile(g$vmr, c(.5, .75, .9), na.rm = TRUE)

 50%   75%   90%
33.9  96.3 109.5

# Variance estimation
g2 <- g %>%
  filter(mean > 0, vmr > 1) %>%
  mutate(size_hat = mean / (vmr - 1))

# confirm the distribution type with fitdistrplus package
fit_pois <- fitdist(df_dyn2$count, "pois")
fit_nb <- fitdist(df_dyn2$count, "nbinom")
plot(fit_pois)

```



```
gofstat(fit_pois)
```

Chi-squared statistic: Inf

Degree of freedom of the Chi-squared distribution: 7

Chi-squared p-value: 0

the p-value may be wrong with some theoretical counts < 5

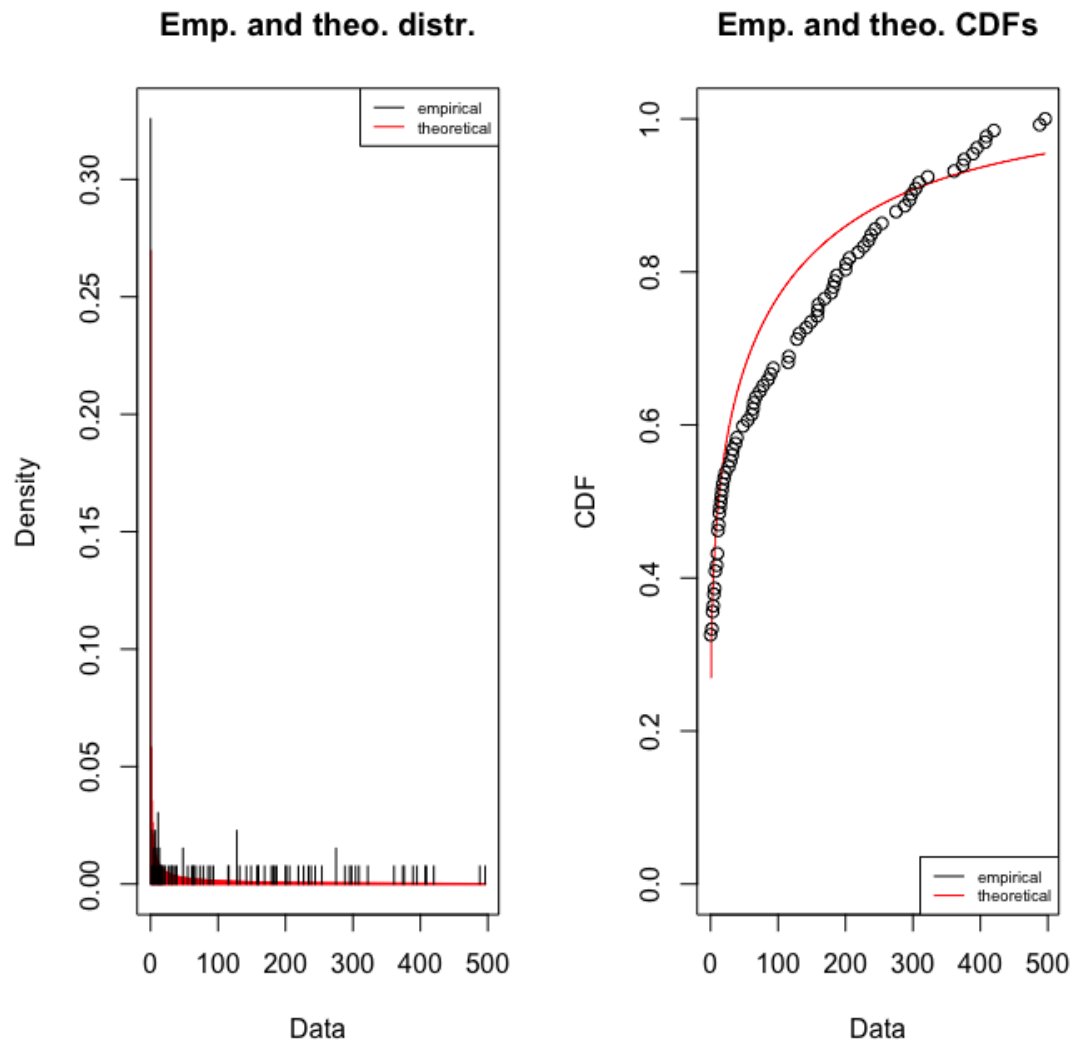
Chi-squared table:

	obscounts	theocounts
<= 0	4.30e+01	7.81e-39
<= 7	1.10e+01	9.79e-29
<= 14	1.10e+01	3.60e-22
<= 37	1.10e+01	5.49e-09
<= 85	1.10e+01	3.06e+01
<= 158	1.10e+01	1.01e+02
<= 219	1.10e+01	3.21e-08
<= 304	1.10e+01	0.00e+00
> 304	1.20e+01	0.00e+00

Goodness-of-fit criteria

	1-mle-pois
Akaike's Information Criterion	22334
Bayesian Information Criterion	22337

```
plot(fit_nb)
```



```
gofstat(fit_nb)
```

Chi-squared statistic: 24.2
Degree of freedom of the Chi-squared distribution: 6
Chi-squared p-value: 0.000488
Chi-squared table:

	obscounts	theocounts
<= 0	43.00	35.58
<= 7	11.00	24.62

```

<= 14      11.00      8.91
<= 37      11.00     14.89
<= 85      11.00     14.48
<= 158     11.00     11.02
<= 219     11.00      5.48
<= 304     11.00      5.00
> 304      12.00     12.00

```

Goodness-of-fit criteria

	1-mle-nbinom
Akaike's Information Criterion	1238
Bayesian Information Criterion	1244

```
exp(0.25)
```

```
[1] 1.28
```

```
# initial size estimation
summary(g2$size_hat)
```

```

      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.185   0.673   1.276   5.291   5.740  48.516

```

```
quantile(g2$size_hat, c(0.25, 0.5, 0.75), na.rm = TRUE)
```

```

      25%   50%   75%
0.673 1.276 5.740

```

Variance-to-mean ratios far exceeding 1, violating Poisson assumptions, also, fitdist function confirms the selection of negative binomial. Therefore, a negative binomial likelihood was used for all subsequent model fitting.

6. Functions for modeling

6.1 Model Fitting

```

get_nll <- function(p, this_data, known, times,
                    model_name = "Geeraerd") {

  # if (this_data$temp_target_c[1] == 74) {
  #   browser()
  # }

  all_pars <- as.list(c(p, known))

  # unlog transformation
  if ("logN0" %in% names(all_pars)) {
    all_pars$logN0 <- 10^all_pars$logN0
    all_pars$logN0 <- NULL
  }
}

```

```

if ("logNmin" %in% names(all_pars)) {
  all_pars$N_min <- 10^all_pars$logNmin
  all_pars$logNmin <- NULL
}

# Negative binomial distribution require size (dispersion) as parameter estimate
size <- exp(all_pars$log_size)
all_pars$log_size <- NULL

# Temp profile for this target temperature condition
this_temp <- this_data %>%
  dplyr::select(time, temperature) %>%
  distinct() %>%
  arrange(time)

# Predicted data
pred <- predict_inactivation(
  simulation_model = model_name,
  times = times,
  parms = all_pars,
  temp_profile = this_temp
)$simulation %>%
  as.data.frame()

# Observed data
obs <- this_data %>%
  dplyr::select(time, count, dilution, plated_ml)

merged <- merge(
  x = obs,
  y = pred[, c("time", "logN")],
  by = "time",
  all.x = TRUE,
  suffixes = c("_obs", "_mod")
)

# if (any(is.na(merged$logN)) || any(!is.finite(merged$logN))) return(Inf)

# Predicted concentration from predicted log10 CFU/ml
pred_N <- 10^merged$logN
pred_count <- pred_N * merged$plated_ml * merged$dilution

# if (any(!is.finite(pred_count)) || any(pred_count <= 0)) return(Inf)
pred_count <- pmax(pred_count, 1e-12)

# NB log-likelihood
ll <- dnbinom(merged$count, mu = pred_count, size = size, log = TRUE)
# if (any(!is.finite(ll))) {

```

```

#   print("out")
#   return(Inf)
# }

# print(sum(ll))

-sum(ll, na.rm = TRUE)
}

get_total_nll <- function(p, split_data, known, times_list,
                        model_name = "Geeraerd") {
  nlls <- mapply(
    FUN = function(dat, times) {
      get_nll(p = p, this_data = dat, known = known, times = times,
             model_name = model_name)},
    dat = split_data,
    times = times_list,
    SIMPLIFY = TRUE
  )

  # if (any(!is.finite(nlls))) return(Inf)

  sum(nlls, na.rm = TRUE)
}

global_fit <- function(d, guess, known, model_name = "Geeraerd", upper = Inf,
                      lower = -Inf, niter = 5000) {
  d2 <- d %>%
    mutate(
      time = as.numeric(time_min),
      temperature = as.numeric(temp_mean),
      time = ifelse(time < 2e-5, 2e-5, time),
      temp_target_c = as.factor(temp_target_c)
    )

  ## Split by target temperature
  split_data <- split(d2, d2$temp_target_c)

  ## Precompute times per condition
  times_list <- lapply(split_data, function(df) {
    df$time %>% unique() %>% sort() %>% na.omit()
  })

  modMCMC(f = get_total_nll, p = guess, split_data = split_data,
         known = known, times_list = times_list, model_name = model_name,
         upper = upper, lower = lower, niter = niter
  )
}

```


##Goodness-of-fit

```
compute_residuals_dyn <- function(data, model_name, bestpar, known_pars) {

  bp <- as.list(bestpar)
  size <- exp(as.numeric(bp$log_size))
  bp$log_size <- NULL

  # Unlog parameters if present
  if ("logN0" %in% names(bp)) {
    bp$N0 <- 10^bp$logN0
    bp$logN0 <- NULL
  }
  if ("logNmin" %in% names(bp)) {
    bp$N_min <- 10^bp$logNmin
    bp$logNmin <- NULL
  }

  d2 <- data %>%
    mutate(
      cond = as.factor(temp_target_c),
      time = round(as.numeric(time_min), 3),
      temperature = as.numeric(temp_mean)
    )

  preds <- d2 %>%
    split(.$cond) %>%
    map_df(function(df) {

      temp_prof <- df %>%
        dplyr::select(time, temperature) %>%
        distinct() %>%
        arrange(time)

      times_use <- sort(unique(df$time))

      sim <- quiet_predict_inactivation(
        simulation_model = model_name,
        times = times_use,
        parms = c(bp, as.list(known_pars)),
        temp_profile = temp_prof
      )$simulation %>%
        as.data.frame() %>%
        mutate(time = round(time, 3))

      data.frame(cond = df$cond, time = sim$time, m = sim$logN)
    }) %>%
    distinct(cond, time, .keep_all = TRUE)
```

```

d2 %>%
  left_join(preds, by = c("cond", "time")) %>%
  mutate(pred_count = (10^m) * plated_ml * dilution,
         pred_count = ifelse(is.finite(pred_count) & pred_count > 0, pred_c
ount, NA_real_))
}

# AIC and AICc calculation from log-likelihood
compute_fitstats <- function(res_df, bestpar, k, comm_df = NULL) {
  size <- exp(bestpar[["log_size"]])

  # If comm_df is provided, restrict to exactly those rows
  if (!is.null(comm_df)) {
    res_df <- res_df %>%
      dplyr::semi_join(
        comm_df %>% dplyr::distinct(cond, time, dilution, plated_ml, count),
        by = c("cond", "time", "dilution", "plated_ml", "count")
      )
  }

  ll <- dnbinom(
    x = res_df$count,
    mu = res_df$pred_count,
    size = size,
    log = TRUE
  )

  logLik <- sum(ll)
  n_all <- nrow(res_df)

  AIC <- 2 * k - 2 * logLik
  AICc <- AIC + (2 * k * (k + 1)) / (n_all - k - 1)

  list(logLik = logLik, AIC = AIC, AICc = AICc, n_used = n_all)
}

```

7. Modeling

##Peleg Weibull - Best fit

```

df_dyn2 <- readRDS(here("Data", "df_dyn2.RDS"))

# Plotting-only data: zero colony observations are modeled as count = 0,
# but displayed below the LOD on log-scale concentration plots.
df_dyn_plot <- df_dyn2 %>%
  mutate(
    no_colonies = count == 0L,

```

```

    log_cfu_plot = if_else(no_colonies, 0.5, as.numeric(log_cfu))
  )

known_pe <- c()

guess_pe <- c(k_b = 0.1, n = 1.0, temp_crit = 50, logN0 = 6, log_size = log(1.9))

lower_pe <- c(k_b = 0.001, n = 0.1, temp_crit = 45, logN0 = 5, log_size = log(0.05))

upper_pe <- c(k_b = 20, n = 5, temp_crit = 65, logN0 = 7, log_size = log(20))

set.seed(5025)

invisible(capture.output(my_model_pe <- global_fit(
  d = df_dyn2, guess = guess_pe, known = known_pe, model_name = "Peleg",
  niter = 5000, lower = lower_pe, upper = upper_pe), type = "output"))

plot(my_model_pe)

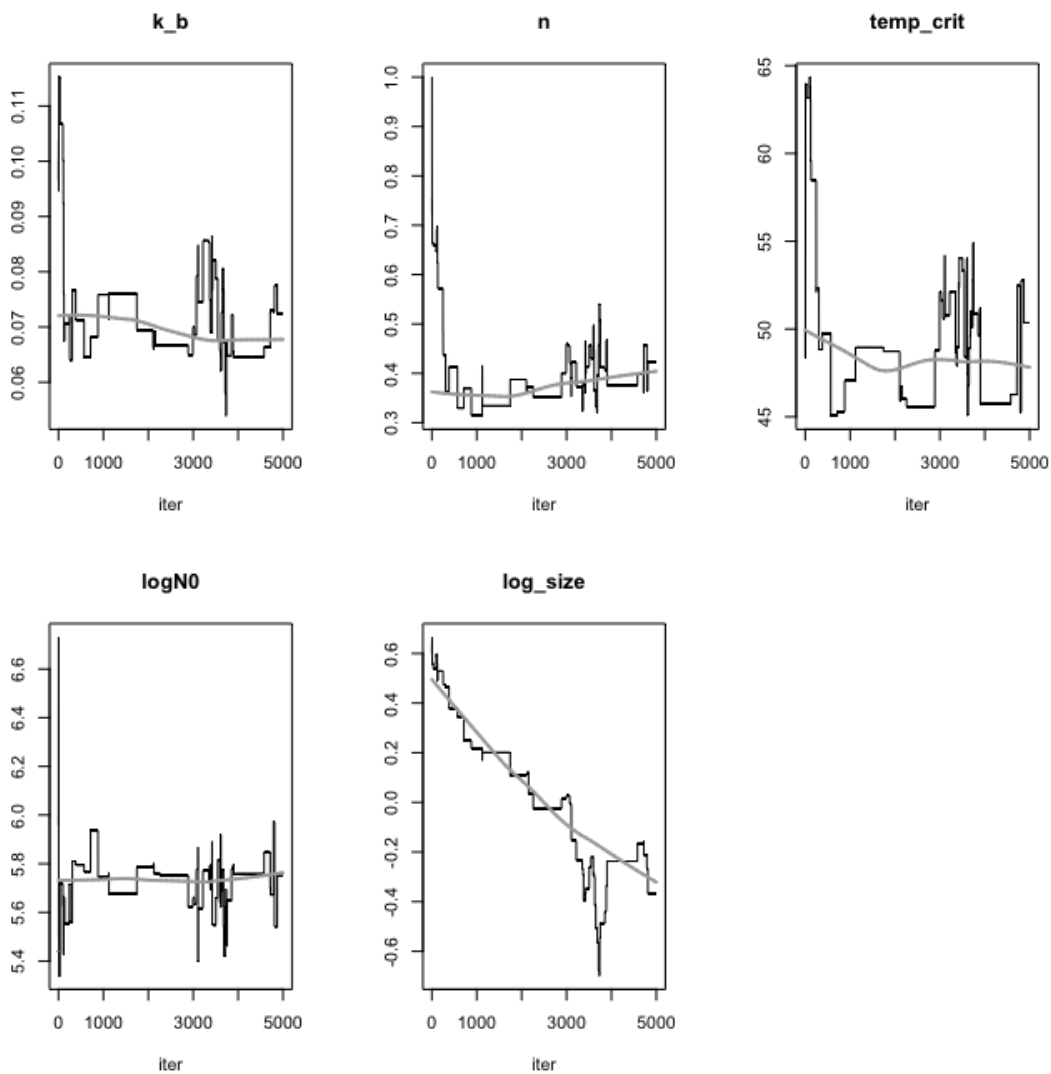
summary(my_model_pe)

      k_b      n temp_crit logN0 log_size
mean 0.07119 0.3884    48.72 5.7276  0.00561
sd    0.00777 0.0665     3.68 0.0919  0.27446
min   0.05394 0.3141    45.06 5.3397 -0.69863
max   0.11522 1.0000    64.35 6.7311  0.66588
q025  0.06632 0.3512    45.74 5.6764 -0.23689
q050  0.06948 0.3766    48.73 5.7510 -0.02567
q075  0.07587 0.4116    49.76 5.7720  0.20006

apply(my_model_pe$pars, 2, quantile, probs = c(0.025, 0.975))

      k_b      n temp_crit logN0 log_size
2.5%  0.0645 0.314    45.1  5.54  -0.487
97.5% 0.0856 0.625    59.6  5.94   0.529

```



7.0.1 Decimal reduction time calculation

```
temps <- c(63, 74, 90, 98)
```

```
pars <- as.data.frame(my_model_pe$pars)
pars$s <- exp(pars$log_size)
mean(pars$s)
```

```
[1] 1.04
```

```
sd(pars$s)
```

```
[1] 0.29
```

```
quantile(pars$s, probs = c(0.025, 0.975))
```

```
2.5% 97.5%
0.615 1.697
```

```

# compute b(T)
calc_b <- function(T, k_b, temp_crit){
  log1p(exp(k_b * (T - temp_crit)))
}

# compute D-value
calc_D <- function(b, n){
  (log(10) / b)^(1 / n)
}

# compute posterior samples
post_samples <- pars %>%
  mutate(
    b63 = calc_b(63, k_b, temp_crit),
    b74 = calc_b(74, k_b, temp_crit),
    b98 = calc_b(98, k_b, temp_crit),

    D63 = calc_D(b63, n),
    D74 = calc_D(b74, n),
    D98 = calc_D(b98, n)
  )

# summarize mean + 95% CrI
summary_table <- post_samples %>%
  summarise(
    b63_mean = mean(b63),
    b63_lwr = quantile(b63, 0.025),
    b63_upr = quantile(b63, 0.975),
    b63_sd = sd(b63),

    b74_mean = mean(b74),
    b74_lwr = quantile(b74, 0.025),
    b74_upr = quantile(b74, 0.975),
    b74_sd = sd(b74),

    b98_mean = mean(b98),
    b98_lwr = quantile(b98, 0.025),
    b98_upr = quantile(b98, 0.975),
    b98_sd = sd(b98),

    D63_mean = mean(D63),
    D63_lwr = quantile(D63, 0.025),
    D63_upr = quantile(D63, 0.975),
    D63_sd = sd(D63),

    D74_mean = mean(D74),
    D74_lwr = quantile(D74, 0.025),
    D74_upr = quantile(D74, 0.975),
    D74_sd = sd(D74),
  )

```

```

    D98_mean = mean(D98),
    D98_lwr = quantile(D98, 0.025),
    D98_upr = quantile(D98, 0.975),
    D98_sd = sd(D98),
  )

```

summary_table

```

    b63_mean b63_lwr b63_upr b63_sd b74_mean b74_lwr b74_upr b74_sd b98_mean
1      1.32  0.815   1.47  0.16    1.94   1.39   2.17  0.172    3.52
    b98_lwr b98_upr b98_sd D63_mean D63_lwr D63_upr D63_sd D74_mean D74_lwr
1      2.85   3.94  0.243    4.32   3.37   5.62  0.655    1.56   1.22
    D74_upr D74_sd D98_mean D98_lwr D98_upr D98_sd
1      2.43  0.263   0.339   0.189   0.689   0.103

```

7.0.2 Prediction of reduction

```
### Predicted log10 reduction from model parameters only
```

```
#| message: false
```

```
#| warning: false
```

```
pars <- as.data.frame(my_model_pe$pars)
```

```
predict_reduction_one_draw <- function(profile_df, draw_row) {
```

```
  p <- as.list(draw_row)
```

```
  p$log_size <- NULL
```

```
  # Arbitrary starting value only used as a reference point.
```

```
  # It cancels out when calculating log10 reduction.
```

```
  p$N0 <- 10^6
```

```
  p$logN0 <- NULL
```

```
  temp_prof <- profile_df %>%
```

```
    dplyr::select(time = time_min, temperature = temp_mean) %>%
```

```
    dplyr::distinct() %>%
```

```
    dplyr::arrange(time)
```

```
  times_use <- seq(0, max(temp_prof$time), by = 0.01)
```

```
  sim <- suppressWarnings(
```

```
    quiet_predict_inactivation(
```

```
      simulation_model = "Peleg",
```

```
      times = times_use,
```

```
      parms = p,
```

```
      temp_profile = temp_prof
```

```
    )$simulation %>%
```

```

    as.data.frame()
  )

  sim %>%
    dplyr::mutate(
      log_reduction = logN[which.min(time)] - logN
    )
}

profiles_split <- split(df_temp, df_temp$temp_target_c)

reduction_all <- purrr::imap_dfr(profiles_split, function(prof, target_name)
{
  purrr::map_dfr(seq_len(nrow(pars)), function(i) {

    predict_reduction_one_draw(
      profile_df = prof,
      draw_row = pars[i, ]
    ) %>%
      dplyr::mutate(
        draw = i,
        temp_target_c = target_name
      )
  })
})

reduction_table_1min <- reduction_all %>%
  dplyr::group_by(temp_target_c, time) %>%
  dplyr::summarise(
    mean_log_reduction = mean(log_reduction, na.rm = TRUE),
    lwr_log_reduction = quantile(log_reduction, 0.025, na.rm = TRUE),
    upr_log_reduction = quantile(log_reduction, 0.975, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  dplyr::filter(abs(time - round(time)) < 1e-8) %>%
  dplyr::arrange(temp_target_c, time)

print(reduction_table_1min, n = Inf, width = Inf)

# A tibble: 32 × 5
  temp_target_c time mean_log_reduction lwr_log_reduction upr_log_reduction
  <chr>         <dbl>         <dbl>         <dbl>         <dbl>
1 63           1         0.348         0.177         0.444
2 63           2         0.778         0.480         0.910
3 63           3         1.19          0.841         1.36
4 63           4         1.54          1.19          1.73
5 63           5         1.84          1.51          2.05
6 63           6         2.10          1.81          2.32

```

7	63	7	2.32	2.04	2.55
8	63	8	2.52	2.24	2.77
9	63	9	2.70	2.42	2.97
10	63	10	2.87	2.58	3.15
11	63	11	3.02	2.76	3.32
12	63	12	3.16	2.89	3.47
13	74	1	0.397	0.203	0.500
14	74	2	0.962	0.602	1.10
15	74	3	1.55	1.13	1.73
16	74	4	2.08	1.68	2.27
17	74	5	2.54	2.21	2.74
18	74	6	2.94	2.69	3.14
19	74	7	3.29	3.01	3.49
20	74	8	3.59	3.30	3.94
21	74	9	3.87	3.56	4.23
22	74	10	4.11	3.80	4.53
23	74	11	4.34	3.98	4.87
24	74	12	4.55	4.15	5.24
25	98	1	0.904	0.488	1.04
26	98	2	2.22	1.67	2.44
27	98	3	3.40	2.95	3.67
28	98	4	4.38	3.94	4.89
29	98	5	5.20	4.71	5.79
30	98	6	5.89	5.30	6.60
31	98	7	6.48	5.75	7.31
32	98	8	7.00	6.14	8.20

7.0.3 Plot using the best parameter set with observed value overlaid

```
# Plotting-only data for observed points.
df_dyn_plot <- df_dyn2 %>%
  mutate(
    no_colonies = count == 0L,
    log_cfu_plot = if_else(no_colonies, 0.5, as.numeric(log_cfu))
  )

df_dyn_fit <- df_dyn_plot %>%
  mutate(
    temp_target_c = as.factor(temp_target_c),
    time = as.numeric(time_min),
    temperature = as.numeric(temp_mean)
  )

# un-log
fit_p_pe <- as.list(my_model_pe$bestpar)

if ("logN0" %in% names(fit_p_pe)) {
  fit_p_pe$logN0 <- 10^fit_p_pe$logN0
  fit_p_pe$logN0 <- NULL
}
```



```

if ("logNmin" %in% names(fit_p_pe)) {
  fit_p_pe$N_min <- 10^fit_p_pe$logNmin
  fit_p_pe$logNmin <- NULL
}

if ("log_size" %in% names(fit_p_pe)) {
  fit_p_pe$log_size <- NULL
}

plots <- df_dyn_fit %>%
  split(.$temp_target_c) %>%
  purrr::map(~{
    temp_prof <- .x %>%
      dplyr::select(time, temperature) %>%
      distinct() %>%
      arrange(time)

    times_pred <- seq(0, max(.$time, na.rm = TRUE), length.out = 200)

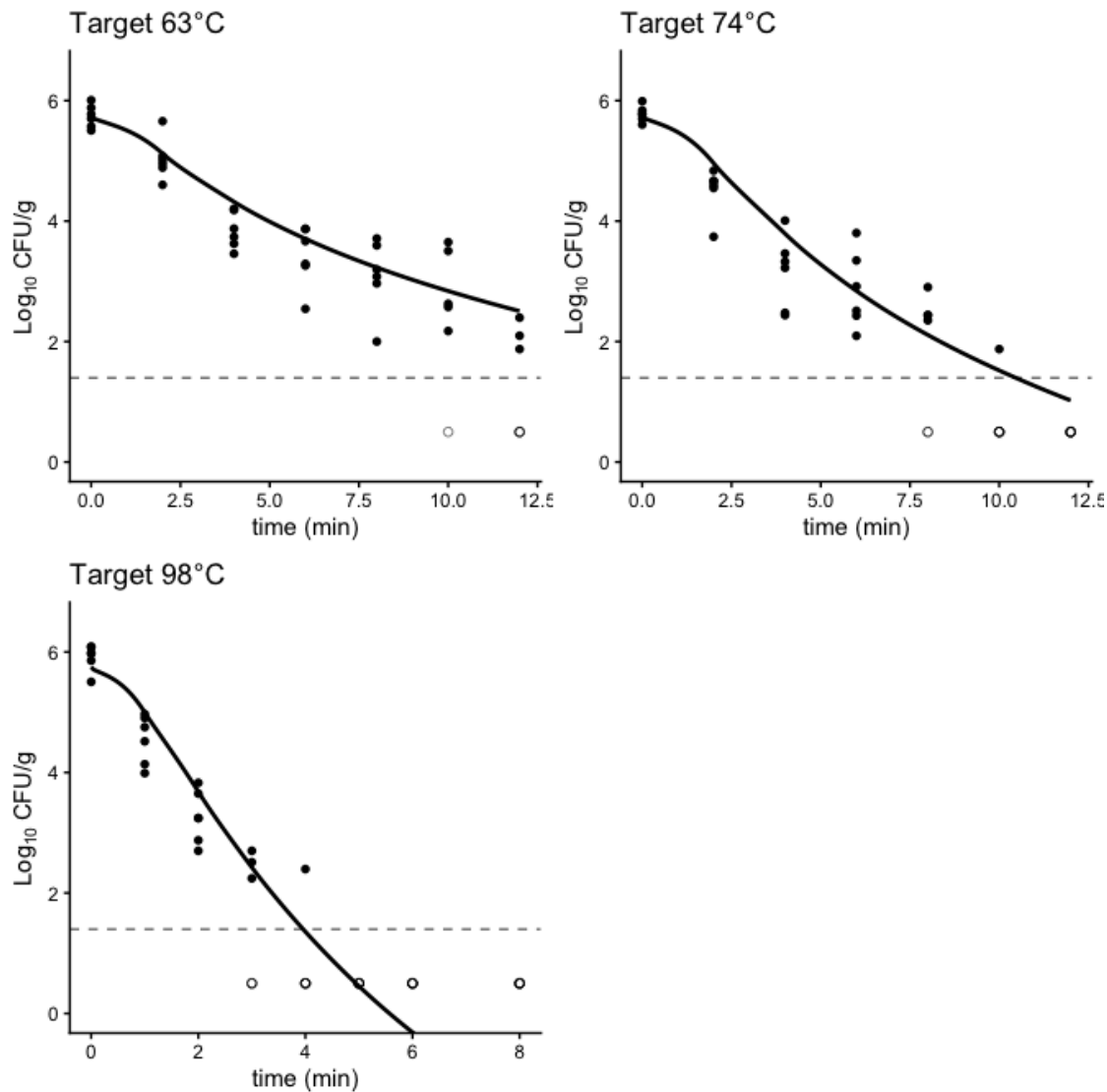
    pred <- quiet_predict_inactivation(
      simulation_model = "Peleg",
      times = times_pred,
      parms = c(fit_p_pe, as.list(known_pe)),
      temp_profile = temp_prof
    )

    sim <- pred$simulation %>%
      as.data.frame() %>%
      mutate(logN_clip = pmax(logN, LOD))

    ggplot() +
      geom_hline(yintercept = LOD, linetype = "dashed", color = "gray50") +
      geom_line(data = sim, aes(x = time, y = logN), linewidth = 0.9) +
      geom_point(data = filter(.x, !no_colonies), aes(x = time, y = log_cfu_p
lot),
      size = 1.6, shape = 16, color = "black") +
      geom_point(data = filter(.x, no_colonies), aes(x = time, y = log_cfu_pl
ot),
      size = 1.6, shape = 1, color = "black", alpha = 0.5) +
      labs(
        title = paste0("Target ", as.character(unique(.$temp_target_c)), "°C
"),
        x = "time (min)",
        y = expression(Log[10]~"CFU/g")
      ) +
      coord_cartesian(ylim = c(0, 6.5)) +
      theme_classic()
  })

```

```
cowplot::plot_grid(plotlist = plots)
```



7.0.4 Goodness-of-fit check

```
df_resid_pe <- compute_residuals_dyn(
  data = df_dyn2,
  model_name = "Peleg",
  bestpar = my_model_pe$bestpar,
  known_pars = known_pe
)
```

```
# fit stats
fitstats_pe <- compute_fitstats(
  res_df = df_resid_pe,
  bestpar = my_model_pe$bestpar,
```

```

    k = 4
  )

fitstats_pe$AIC
[1] 1087

fitstats_pe$AICc
[1] 1088

```

7.0.5 Bacterial reduction estimation given different initial conc.

7.1 Plot for publication

```

make_interv_pe <- function(m, data, known, model = "Peleg") {

  p <- unlist(m)
  p <- p[!names(p) %in% c("log_size", "i")]
  p["N0"] <- 10^p["logN0"]

  data %>%
    split(.$temp_target_c) %>%
    map(~{
      dat <- .x %>%
        mutate(
          time = as.numeric(time_min),
          temperature = as.numeric(temp_mean)
        )

      temp_prof <- dat %>%
        dplyr::select(time, temperature) %>%
        distinct() %>%
        arrange(time)

      times_pred <- seq(0, max(dat$time, na.rm = TRUE), length.out = 200)

      predict_inactivation(
        simulation_model = model,
        times = times_pred,
        parms = c(p, known),
        temp_profile = temp_prof
      )$simulation %>%
        as.data.frame()
    }) %>%
    imap_dfr(~ mutate(.x, temp_target_c = .y))
}

## Posterior predictions
MCMC_preds_pe <- my_model_pe$pars %>%

```

```

as_tibble() %>%
slice_sample(n = 1000, replace = TRUE) %>%
mutate(i = row_number()) %>%
split(.$i) %>%
purrr::map_dfr(
  ~ {
    invisible(capture.output(
      pred <- make_interv_pe(.x, data = df_dyn2, known = known_pe, model =
"Peleg"),
      type = "output"
    ))
    pred
  },
  .id = "i"
) %>%
summarise(
  m = median(logN, na.rm = TRUE),
  q2.5 = quantile(logN, 0.025, na.rm = TRUE),
  q97.5 = quantile(logN, 0.975, na.rm = TRUE),
  .by = c("temp_target_c", "time")
)

## rescale temperature to match CFU axis
temp_scaled <- df_temp %>%
  mutate(temp_target_c=factor(temp_target_c),
         time=as.numeric(time_min),
         scal_temp=(temp_mean-20)/(100-20)*6)

# Plotting-only data for observed points.
df_dyn_plot <- df_dyn2 %>%
  mutate(
    no_colonies = count == 0L,
    log_cfu_plot = if_else(no_colonies, 0.5, as.numeric(log_cfu))
  )

make_panel <- function(Ttarget, xlim=NULL, xbreaks=NULL,
                      show_left_axis=FALSE,
                      show_right_axis=FALSE,
                      show_temp_title=FALSE){

  ## model predictions (clip at 0 for plotting)
  pred_sub <- MCMC_preds_pe %>%
    filter(temp_target_c==Ttarget) %>%
    mutate(m_plot=pmax(m,0),
           q2.5_plot=pmax(q2.5,0),
           q97.5_plot=pmax(q97.5,0))

  ## observed data
  dat_sub <- df_dyn_plot %>%

```

```

filter(temp_target_c==Ttarget) %>%
mutate(time=as.numeric(time_min),
       logN=as.numeric(log_cfu_plot),
       strain=factor(strain))

## temperature profile (already rescaled)
temp_sub <- temp_scaled %>%
  filter(temp_target_c==Ttarget)

ggplot() +
  ## uncertainty ribbon (95% CI)
  geom_ribbon(data=pred_sub,
            aes(x=time,ymin=q2.5_plot,ymax=q97.5_plot),
            fill="gray80",alpha=0.8) +

  ## median prediction line
  geom_line(data=pred_sub,
            aes(x=time,y=m_plot),
            linewidth=0.9,color="black") +

  ## observed points; no-colony observations are plotted below the LOD line
  geom_point(data=filter(dat_sub,!no_colonies),
            aes(x=time,y=logN,shape=strain),
            size=1.8,stroke=0.5,color="black",fill="black") +
  geom_point(data=filter(dat_sub,no_colonies),
            aes(x=time,y=logN),
            shape=1,size=1.8,stroke=0.5,color="black",alpha=0.5) +

  ## temperature curve (scaled to same axis)
  geom_line(data=temp_sub,
            aes(x=time,y=scal_temp),
            linetype="longdash",linewidth=0.6,color="gray35") +

  ## LOD horizontal line
  geom_hline(yintercept=LOD,
            linetype="dashed",linewidth=0.6,color="gray45") +

  ## shape mapping for strains
  scale_shape_manual(values=c(16,2)) +

  ## primary + optional secondary axis
  scale_y_continuous(
    name=if(show_left_axis) expression(Log[10]~"CFU/g") else NULL, # left a
axis only for first panel
    breaks=seq(0,6,1),
    sec.axis=if(show_right_axis) sec_axis(                                # right
axis only for last panel
    ~./6*80+20,
    breaks=seq(20,100,20),

```

```

    name=if(show_temp_title) "Temperature (°C)" else NULL
  ) else waiver()
) +

## x-axis
scale_x_continuous(breaks=xbreaks) +

## limits
coord_cartesian(xlim=xlim,ylim=c(0,6)) +

labs(x= "Time (min)",title=paste0("Target ",as.character(Ttarget),"°C"))
+

## styling
theme_bw(base_size=12) +
theme(
  plot.title=element_text(size=10,face="bold",hjust=0.5),
  axis.text.x=element_text(size=12,color="black"),
  ## left axis visibility control
  axis.text.y.left=if(show_left_axis) element_text(size=12,color="black")
else element_blank(),
  axis.ticks.y.left=if(show_left_axis) element_line(color="black") else element_blank(),
  axis.title.y.left=if(show_left_axis) element_text(size=12) else element_blank(),

  ## right axis visibility control
  axis.text.y.right=if(show_right_axis) element_text(size=12,color="gray25") else element_blank(),
  axis.ticks.y.right=if(show_right_axis) element_line(color="gray25") else element_blank(),
  axis.title.y.right=if(show_right_axis && show_temp_title)
    element_text(size=12,margin=margin(l=6)) else element_blank(),

  ## remove grids + legend
  panel.grid.minor=element_blank(),
  panel.grid.major=element_blank(),
  panel.border=element_rect(linewidth=0.6,color="black"),
  legend.position="none"
)
}

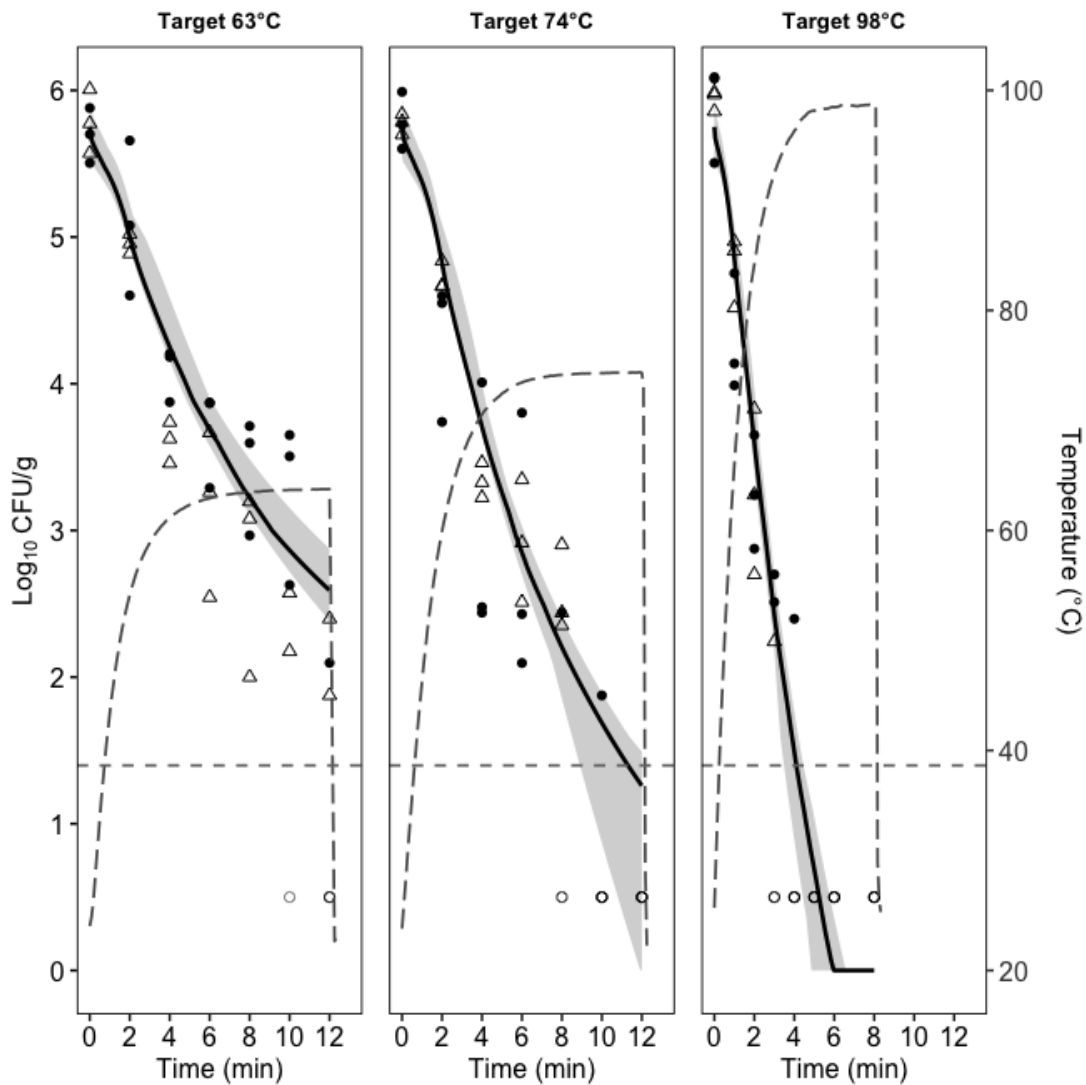
## build panels
p1 <- make_panel("63", xlim=c(0,13), xbreaks=seq(0,12,2), show_left_axis=TRUE)
p2 <- make_panel("74", xlim=c(0,13), xbreaks=seq(0,12,2))
p3 <- make_panel("98", xlim=c(0,13), xbreaks=seq(0,12,2), show_right_axis=TRUE, show_temp_title=TRUE)

## combine (tight layout)

```

```
p_all <- p1 + p2 + p3 + plot_layout(ncol=3)
```

```
p_all
```



```
## save
ggsave(filename=here("Plot", "Vibrio_Oys_PE_Plot.tiff"),
        plot=p_all, width=10, height=4, units="in",
        dpi=1000, bg="white")
```

7.2 Geeraerd

```
model_name <- "Geeraerd"
known_ge_1 <- c(temp_ref = 63)
lower_ge_1 <- c(D_R = .3, z = 2, logN0 = 5, logNmin = -1, log_size = log(0.05),
               C_c0 = 1e-6)
upper_ge_1 <- c(D_R = 6, z = 15, logN0 = 7, logNmin = 1.7, log_size = log(20),
               C_c0 = 1e3)
```

```
# Initial guess from the inactivation data plot
```

```
guess_ge_1 <- c(D_R = 2.4, z = 6, logN0 = 6, logNmin = 0, log_size = log(1.9),  
  C_c0 = 1)
```

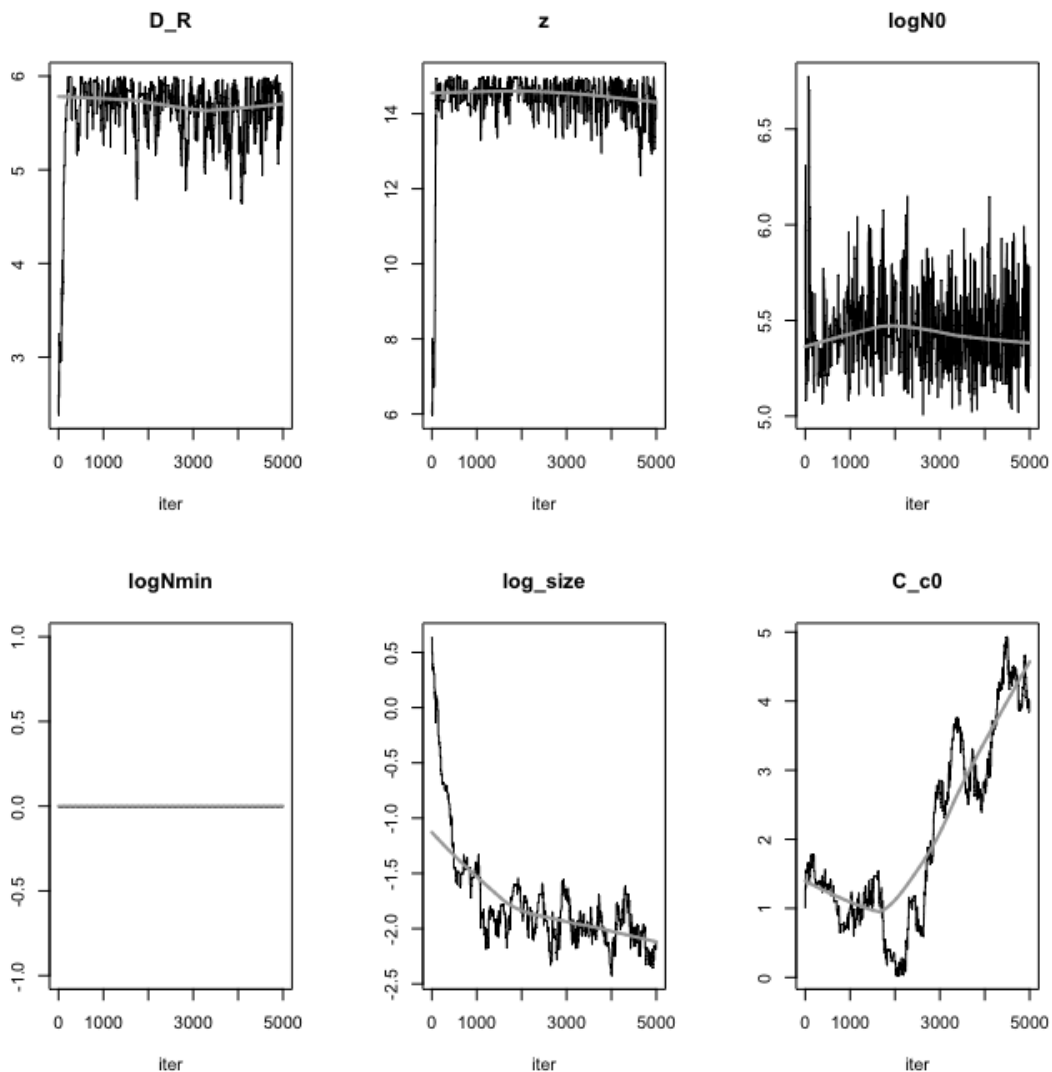
```
set.seed(5025)
```

```
my_model_ge_1 <- global_fit(df_dyn2, guess_ge_1, known_ge_1, niter = 5000,  
  lower = lower_ge_1,  
  upper = upper_ge_1)
```

number of accepted runs: 729 out of 5000 (14.6%)

```
saveRDS(my_model_ge_1, here("Result", "model_Geeraerd_1.RDS"))
```

```
plot(my_model_ge_1)
```




```
apply(my_model_ge_1$pars, 2, quantile, probs = c(0.025, 0.975))
```

	D_R	z	logN0	logNmin	log_size	C_c0
2.5%	4.68	13.1	5.12	0	-2.2871	0.134
97.5%	5.99	15.0	5.90	0	-0.0201	4.566

```
summary(my_model_ge_1)
```

	D_R	z	logN0	logNmin	log_size	C_c0
mean	5.61	14.369	5.447	0	-1.754	2.0446
sd	0.45	0.974	0.207	0	0.497	1.3420
min	2.38	5.970	5.005	0	-2.425	0.0261
max	6.00	15.000	6.776	0	0.642	4.9321
q025	5.47	14.246	5.296	0	-2.047	0.9842
q050	5.73	14.572	5.421	0	-1.894	1.5319
q075	5.87	14.812	5.560	0	-1.630	3.0707

From the summary, the logNmin is unidentifiable in the censored-likelihood formulation, so MCMC pins it to the boundary 0. Let's try put N_min as the known value 10^0

```
model_name <- "Geeraerd"
known_ge <- c(temp_ref = 63, N_min = 10^0)
lower_ge <- c(D_R = .3, z = 2, logN0 = 5, log_size = log(0.05), C_c0 = 1e-6)
upper_ge <- c(D_R = 6, z = 15, logN0 = 7, log_size = log(20), C_c0 = 1e3)
# Initial guess from the inactivation data plot
guess_ge <- c(D_R = 2.4, z = 6, logN0 = 6, log_size = log(1.9), C_c0 = 1)

my_model_ge <- global_fit(df_dyn2, guess_ge, known_ge, niter = 5000,
                          lower = lower_ge,
                          upper = upper_ge)
```

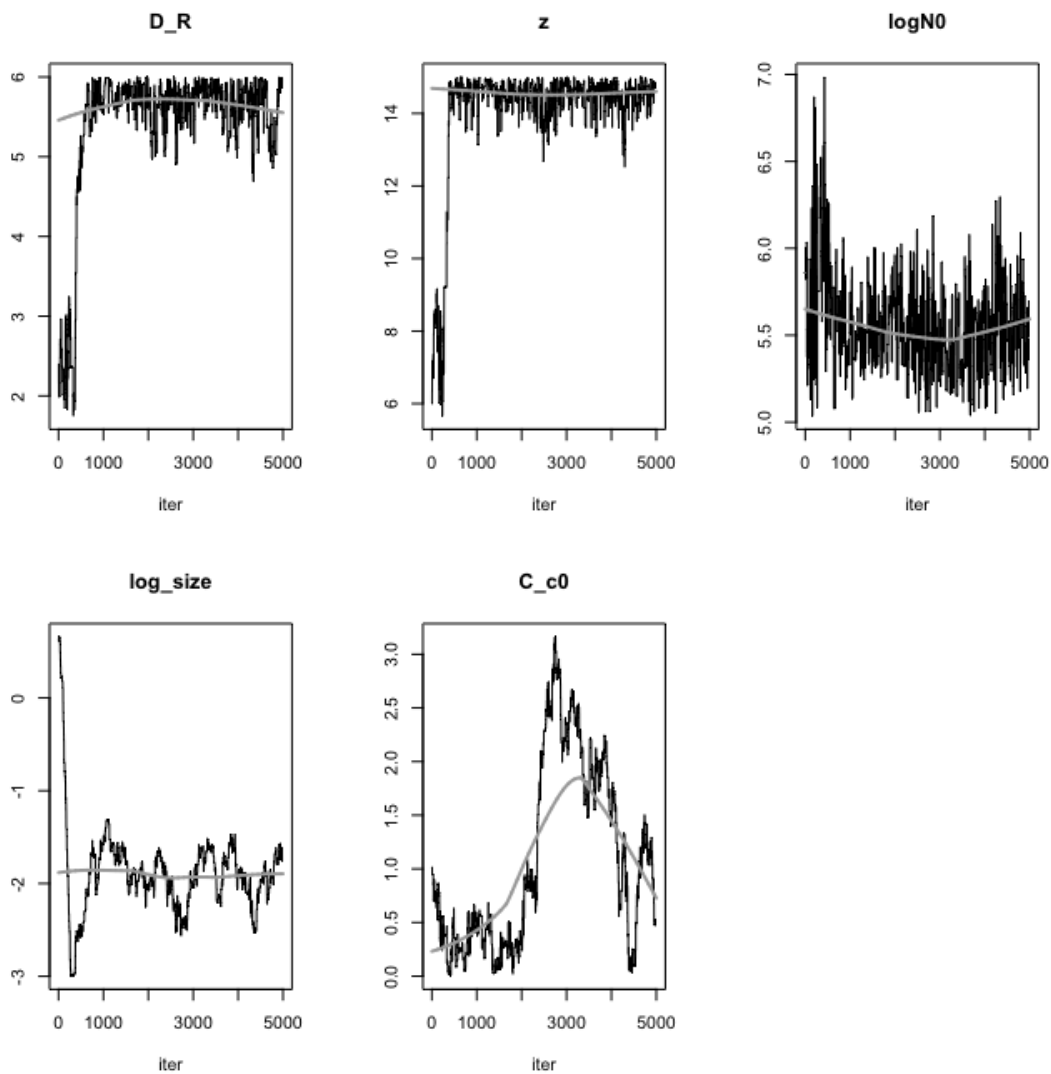
number of accepted runs: 763 out of 5000 (15.3%)

```
saveRDS(my_model_ge, here("Result", "model_Geeraerd.RDS"))
```

```
plot(my_model_ge)
```

```
summary(my_model_ge)
```

	D_R	z	logN0	log_size	C_c0
mean	5.393	14.04	5.552	-1.886	1.09298
sd	0.921	1.75	0.263	0.480	0.86428
min	1.751	5.67	5.037	-2.994	0.00376
max	5.998	15.00	6.984	0.655	3.15972
q025	5.368	14.20	5.358	-2.099	0.33279
q050	5.687	14.56	5.522	-1.904	0.81137
q075	5.878	14.80	5.697	-1.688	1.86726



7.3 Geeraerd with no shoulder

```

model_name <- "Geeraerd"
known_ge_ns <- c(temp_ref = 63, N_min = 10^0, C_c0 = 1e-3)
lower_ge_ns <- c(D_R = .3, z = 2, logN0 = 5, log_size = log(0.05))
upper_ge_ns <- c(D_R = 6, z = 15, logN0 = 7, log_size = log(20))
# Initial guess from the inactivation data plot
guess_ge_ns <- c(D_R = 2.4, z = 6, logN0 = 6, log_size = log(1.9))

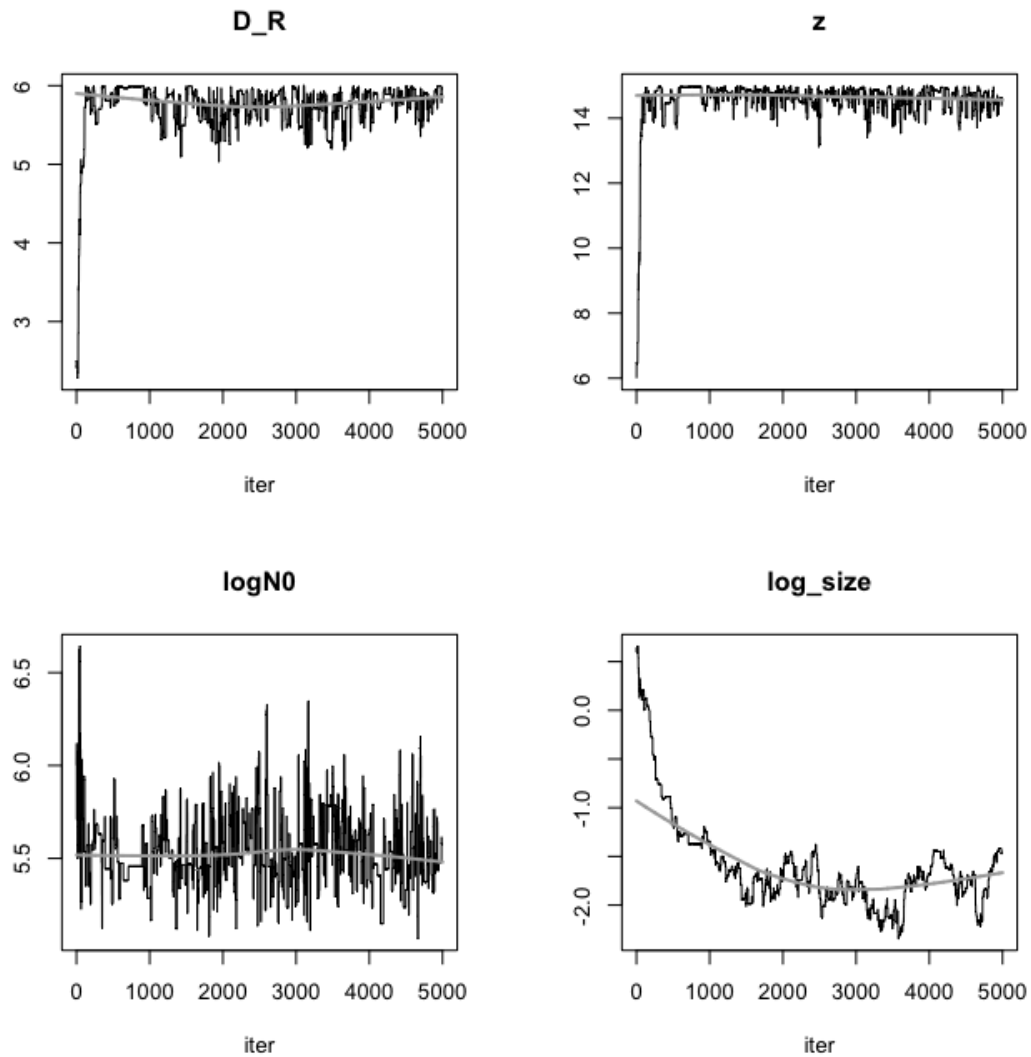
set.seed(5025)
my_model_ge_ns <- global_fit(df_dyn2, guess_ge_ns, known_ge_ns, niter = 5000,
                             lower = lower_ge_ns,
                             upper = upper_ge_ns)

```

number of accepted runs: 554 out of 5000 (11.1%)

```
saveRDS(my_model_ge_ns, here("Result", "model_Geeraerd_no_shoulder.RDS"))
```

```
plot(my_model_ge_ns)
```



```
summary(my_model_ge_ns)
```

	D_R	z	logN0	log_size
mean	5.728	14.562	5.539	-1.596
sd	0.344	0.727	0.191	0.493
min	2.280	6.000	5.068	-2.345
max	6.000	14.999	6.642	0.658
q025	5.619	14.439	5.426	-1.877
q050	5.805	14.718	5.507	-1.702
q075	5.925	14.876	5.637	-1.462

7.3.1 Plot using the best parameter set with observed value overlaid (with shoulder)

```
# un-log
fit_p_ge <- as.list(my_model_ge$bestpar)

if ("logN0" %in% names(fit_p_ge)) {
  fit_p_ge$logN0 <- 10^fit_p_ge$logN0
  fit_p_ge$logN0 <- NULL
}

if ("logNmin" %in% names(fit_p_ge)) {
  fit_p_ge$logN_min <- 10^fit_p_ge$logNmin
  fit_p_ge$logNmin <- NULL
}

if ("log_size" %in% names(fit_p_ge)) {
  fit_p_ge$log_size <- NULL
}

plots <- df_dyn_fit %>%
  split(.$temp_target_c) %>%
  purrr::map(~{
    temp_prof <- .x %>%
      dplyr::select(time, temperature) %>%
      distinct() %>%
      arrange(time)

    times_pred <- seq(0, max(.$time, na.rm = TRUE), length.out = 200)

    pred <- quiet_predict_inactivation(
      simulation_model = "Geeraerd",
      times = times_pred,
      parms = c(fit_p_ge, as.list(known_ge)),
      temp_profile = temp_prof
    )

    sim <- pred$simulation %>%
      as.data.frame() %>%
      mutate(logN_clip = pmax(logN, LOD))

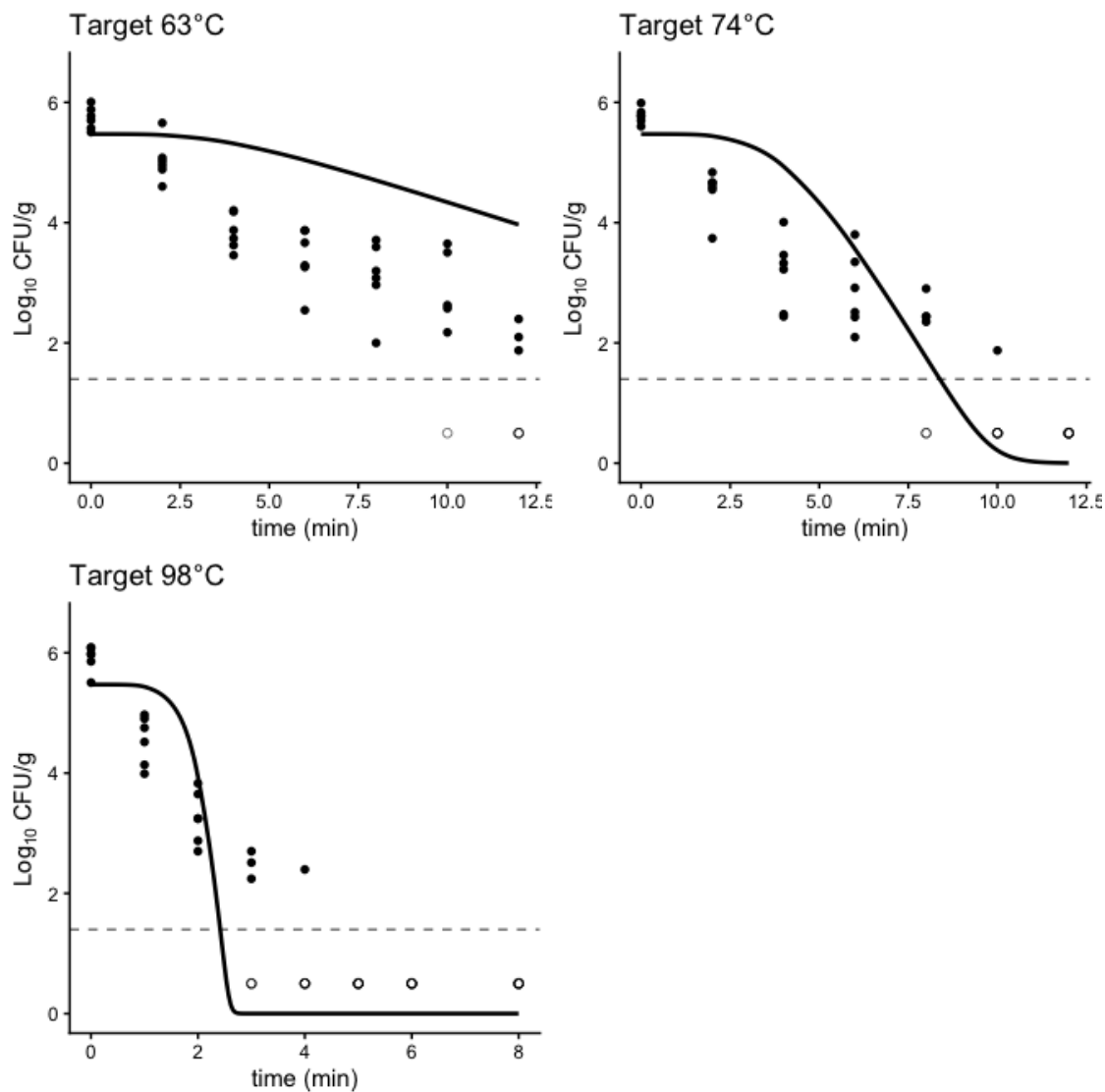
    ggplot() +
      geom_hline(yintercept = LOD, linetype = "dashed", color = "gray50") +
      geom_line(data = sim, aes(x = time, y = logN), linewidth = 0.9) +
      geom_point(data = filter(.$, !no_colonies), aes(x = time, y = log_cfu_plo
t),
                size = 1.6, shape = 16, color = "black") +
      geom_point(data = filter(.$, no_colonies), aes(x = time, y = log_cfu_plo
t),
```

```

      size = 1.6, shape = 1, color = "black", alpha = 0.5) +
labs(
  title = paste0("Target ", as.character(unique(.x$temp_target_c)), "°C"),
  x = "time (min)",
  y = expression(Log[10]~"CFU/g")
) +
coord_cartesian(ylim = c(0, 6.5)) +
theme_classic()
})

```

```
cowplot::plot_grid(plotlist = plots)
```



7.3.2 Plot using the best parameter set with observed value overlaid (with no shoulder)

```

# un-log
fit_p_ge_ns <- as.list(my_model_ge_ns$bestpar)

```

```

if ("logN0" %in% names(fit_p_ge_ns)) {
  fit_p_ge_ns$logN0 <- 10^fit_p_ge_ns$logN0
  fit_p_ge_ns$logN0 <- NULL
}

if ("logNmin" %in% names(fit_p_ge_ns)) {
  fit_p_ge_ns$logNmin <- 10^fit_p_ge_ns$logNmin
  fit_p_ge_ns$logNmin <- NULL
}

if ("log_size" %in% names(fit_p_ge_ns)) {
  fit_p_ge_ns$log_size <- NULL
}

plots <- df_dyn_fit %>%
  split(.$temp_target_c) %>%
  purrr::map(~{
    temp_prof <- .x %>%
      dplyr::select(time, temperature) %>%
      distinct() %>%
      arrange(time)

times_pred <- seq(0, max(.$time, na.rm = TRUE), length.out = 200)

pred <- quiet_predict_inactivation(
  simulation_model = "Geeraerd",
  times = times_pred,
  parms = c(fit_p_ge_ns, as.list(known_ge_ns)),
  temp_profile = temp_prof
)

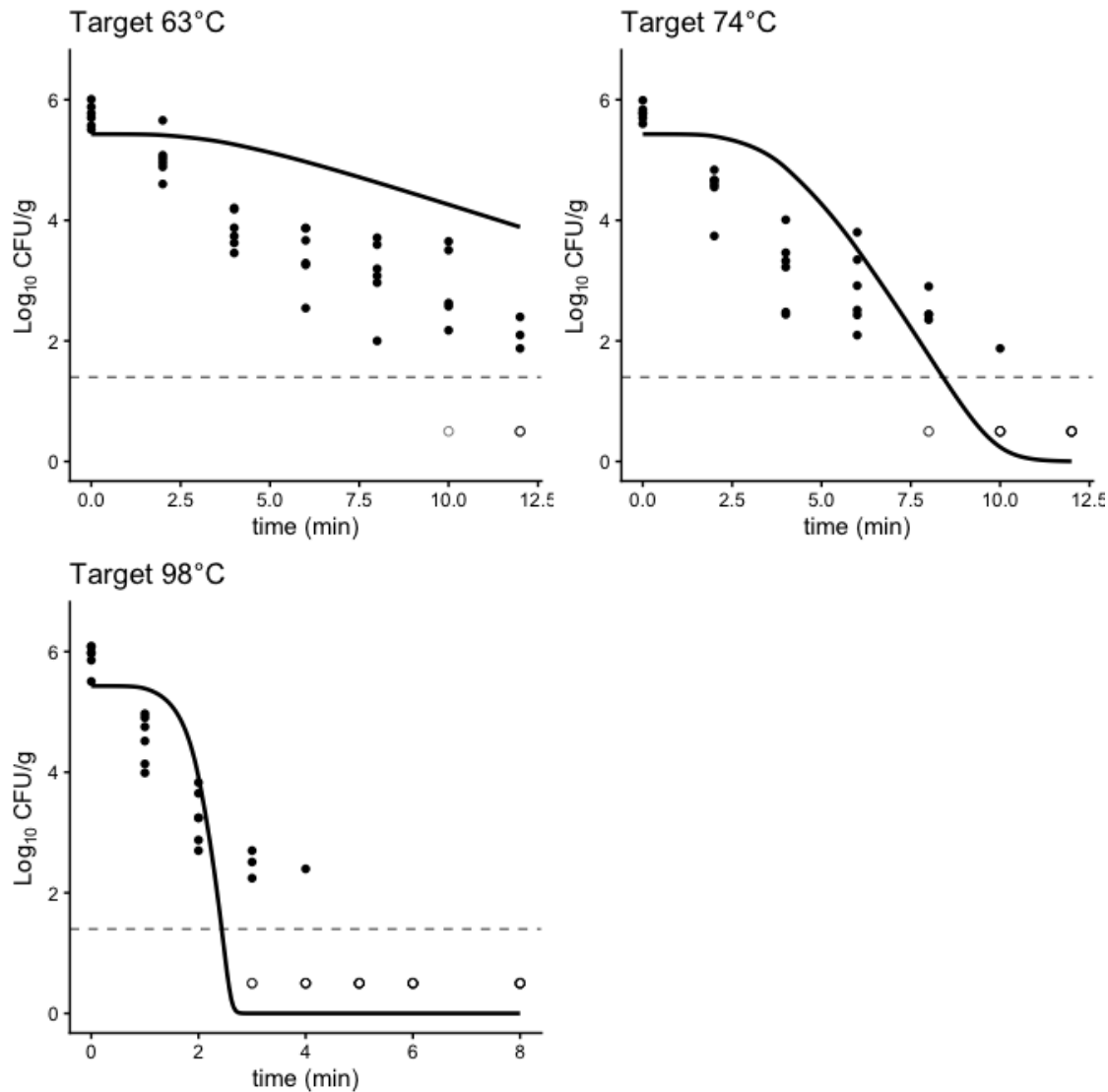
sim <- pred$simulation %>%
  as.data.frame() %>%
  mutate(logN_clip = pmax(logN, LOD))

ggplot() +
  geom_hline(yintercept = LOD, linetype = "dashed", color = "gray50") +
  geom_line(data = sim, aes(x = time, y = logN), linewidth = 0.9) +
  geom_point(data = filter(.$, !no_colonies), aes(x = time, y = log_cfu_plo
t),
    size = 1.6, shape = 16, color = "black") +
  geom_point(data = filter(.$, no_colonies), aes(x = time, y = log_cfu_plo
t),
    size = 1.6, shape = 1, color = "black", alpha = 0.5) +
  labs(
    title = paste0("Target ", as.character(unique(.$temp_target_c)), "°C"),
    x = "time (min)",
    y = expression(Log[10]~"CFU/g")
  )

```

```
) +  
  coord_cartesian(ylim = c(0, 6.5)) +  
  theme_classic()  
})
```

```
cowplot::plot_grid(plotlist = plots)
```



7.3.3 Goodness-of-fit check (with shoulder)

```
df_resid_ge <- compute_residuals_dyn(  
  data = df_dyn2,  
  model_name = "Geeraerd",  
  bestpar = my_model_ge$bestpar,  
  known_pars = known_ge  
)
```

```
# fit stats
fitstats_ge <- compute_fitstats(
  res_df = df_resid_ge,
  bestpar = my_model_ge$bestpar,
  k = 4
)
```

```
fitstats_ge$AIC
```

```
[1] 1357
```

```
fitstats_ge$AICc
```

```
[1] 1358
```

7.3.4 Goodness-of-fit check (with no shoulder)

```
df_resid_ge_ns <- compute_residuals_dyn(
  data = df_dyn2,
  model_name = "Geeraerd",
  bestpar = my_model_ge_ns$bestpar,
  known_pars = known_ge_ns
)
```

```
# fit stats
fitstats_ge_ns <- compute_fitstats(
  res_df = df_resid_ge_ns,
  bestpar = my_model_ge_ns$bestpar,
  k = 4
)
```

```
fitstats_ge_ns$AIC
```

```
[1] 1355
```

```
fitstats_ge_ns$AICc
```

```
[1] 1355
```

7.4 Log-linear (Bigelow)

```
known_b <- c(temp_ref = 63)
```

```
guess_b <- c(D_R = 2.5, z = 6, logN0 = 6, log_size = log(1.9))
```

```
set.seed(5025)
```

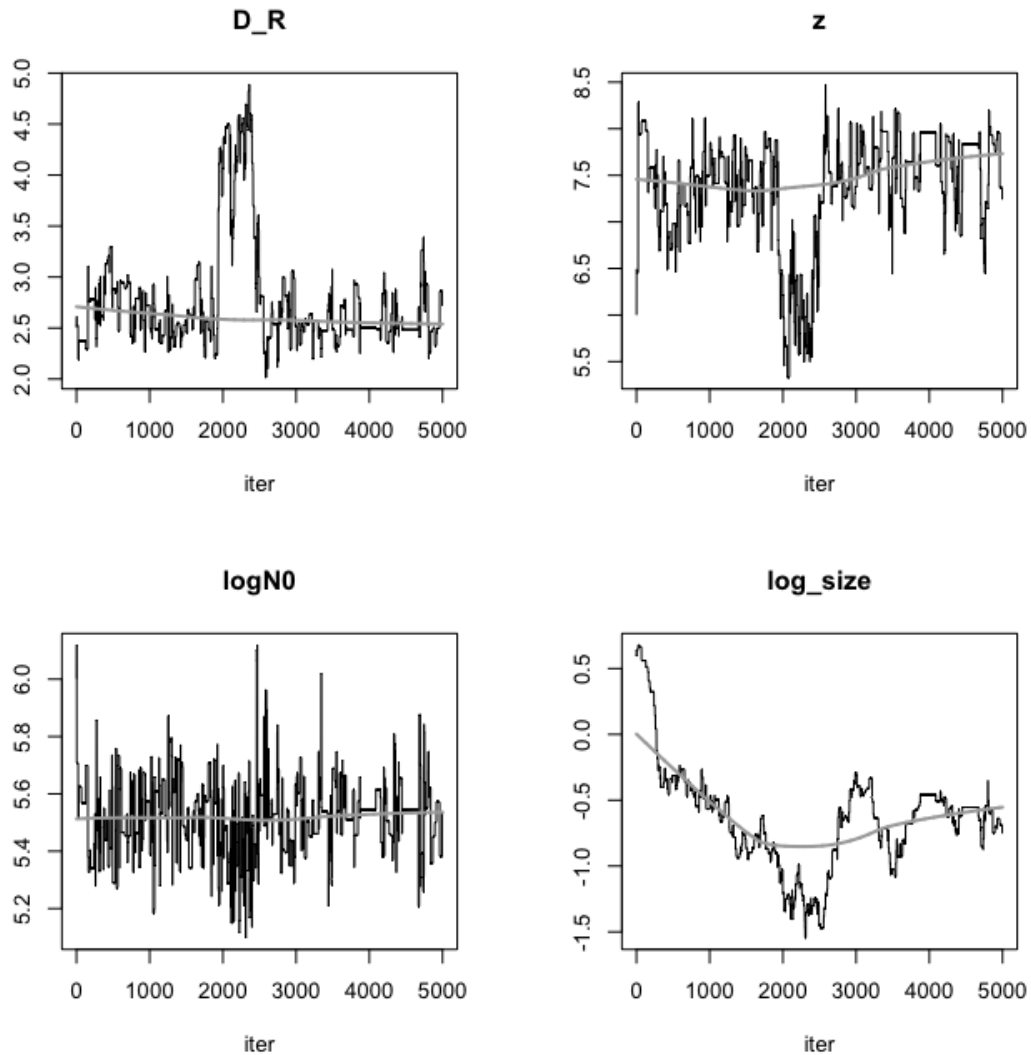
```
my_model_b <- global_fit(df_dyn2, guess_b, known_b, model_name = "Bigelow",
  niter = 5000,
  lower = c(D_R = 2, z = 5, logN0 = 5, log_size = log(0.
05)),
```



```
upper = c(D_R = 20, z = 15, logN0 = 7, log_size = log
(20)))
```

number of accepted runs: 393 out of 5000 (7.86%)

```
plot(my_model_b) # check convergence
```



```
summary(my_model_b)
```

	D_R	z	logN0	log_size
mean	2.770	7.371	5.520	-0.642
sd	0.543	0.588	0.131	0.393
min	2.018	5.332	5.098	-1.544
max	4.886	8.473	6.119	0.677
q025	2.482	7.164	5.443	-0.868
q050	2.565	7.482	5.531	-0.619
q075	2.861	7.832	5.597	-0.458

7.4.1 Plot using the best parameter set with observed value overlaid

```
# un-log
fit_p_b <- as.list(my_model_b$bestpar)

if ("logN0" %in% names(fit_p_b)) {
  fit_p_b$logN0 <- 10^fit_p_b$logN0
  fit_p_b$logN0 <- NULL
}

if ("logNmin" %in% names(fit_p_b)) {
  fit_p_b$logN_min <- 10^fit_p_b$logNmin
  fit_p_b$logNmin <- NULL
}

if ("log_size" %in% names(fit_p_b)) {
  fit_p_b$log_size <- NULL
}

plots <- df_dyn_fit %>%
  split(.$temp_target_c) %>%
  purrr::map(~{
    temp_prof <- .x %>%
      dplyr::select(time, temperature) %>%
      distinct() %>%
      arrange(time)

times_pred <- seq(0, max(.$time, na.rm = TRUE), length.out = 200)

pred <- quiet_predict_inactivation(
  simulation_model = "Bigelow",
  times = times_pred,
  parms = c(fit_p_b, as.list(known_b)),
  temp_profile = temp_prof
)

sim <- pred$simulation %>%
  as.data.frame() %>%
  mutate(logN_clip = pmax(logN, LOD))

ggplot() +
  geom_hline(yintercept = LOD, linetype = "dashed", color = "gray50") +
  geom_line(data = sim, aes(x = time, y = logN), linewidth = 0.9) +
  geom_point(data = filter(.$, !no_colonies), aes(x = time, y = log_cfu_plo
t),
    size = 1.6, shape = 16, color = "black") +
  geom_point(data = filter(.$, no_colonies), aes(x = time, y = log_cfu_plo
t),
    size = 1.6, shape = 1, color = "black", alpha = 0.5) +
```

```

labs(
  title = paste0("Target ", as.character(unique(.x$temp_target_c)), "°C"),
  x = "time (min)",
  y = expression(Log[10]~"CFU/g")
) +
coord_cartesian(ylim = c(0, 6.5)) +
theme_classic()
})

```

```
cowplot::plot_grid(plotlist = plots)
```

7.4.2 Goodness-of-fit check

```

df_resid_b <- compute_residuals_dyn(
  data = df_dyn2,
  model_name = "Bigelow",
  bestpar = my_model_b$bestpar,
  known_pars = known_b
)

```

NA prediction was produced

```

df_resid_b %>%
  dplyr::filter(!is.finite(pred_count) | is.na(pred_count) | pred_count <= 0)

```